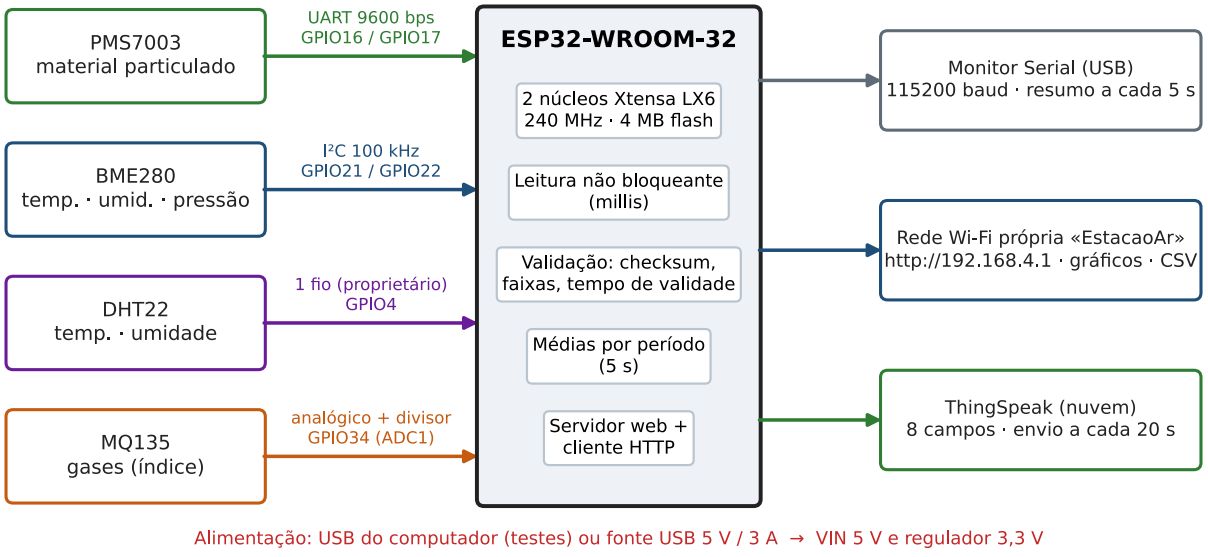
RELATÓRIO TÉCNICO-ACADÊMICO · PROTÓTIPO

Instituição: _____ Curso / disciplina: _____

Estação de Monitoramento da Qualidade do Ar com ESP32

Desenvolvimento do protótipo: fundamentação teórica, montagem, programação e resultados

Guia completo para reproduzir, do zero, uma estação com ESP32-WROOM-32, PMS7003, BME280, DHT22 e MQ135, com página Wi-Fi própria e envio de dados ao ThingSpeak.



Autoria: _____ Outubro de 2026

Resumo

Este trabalho documenta, do zero, o desenvolvimento do protótipo de uma estação de monitoramento da qualidade do ar baseada no microcontrolador ESP32 (módulo ESP32-WROOM-32) e em quatro sensores de baixo custo: o PMS7003, que mede material particulado (PM1,0, PM2,5 e PM10) por espalhamento de laser; o BME280, que mede temperatura, umidade relativa e pressão atmosférica; o DHT22, que mede temperatura e umidade; e o MQ135, um sensor de gases do tipo semicondutor de óxido metálico. São apresentados a fundamentação teórica dos fenômenos físicos medidos, dos circuitos de interface (divisor de tensão, resistores de *pull-up*, níveis lógicos e conversão analógico-digital) e dos protocolos de comunicação (UART, I²C e protocolo de um fio). A montagem seguiu uma metodologia incremental, em dez etapas, com um sensor por vez e verificação elétrica com multímetro antes de cada energização. O documento descreve a preparação do ambiente de desenvolvimento em Linux (Ubuntu 24.04, Arduino IDE e pacote *esp32* 2.0.17), as bibliotecas utilizadas, o mapa de ligações, os programas de teste de cada etapa e o programa final, que lê os quatro sensores de forma não bloqueante, valida os dados, publica uma página web própria em uma rede Wi-Fi criada pelo ESP32 e envia as medições à plataforma ThingSpeak. Os resultados obtidos em laboratório mostram os quatro sensores operando simultaneamente sem falhas de comunicação, concordância entre BME280 e DHT22 dentro das tolerâncias de fábrica (0,9 °C e cerca de 5 a 8 pontos percentuais de umidade), concentração de PM2,5 de cerca de 94 µg/m³ no ambiente do laboratório e resposta do MQ135 de +44 % na exposição a vapor de álcool. Também são discutidas as dificuldades encontradas, as limitações dos sensores de baixo custo e os próximos passos, restritos ao protótipo.

Palavras-chave: qualidade do ar; ESP32; material particulado; sensores de baixo custo; Internet das Coisas.

Abstract

This work documents, from scratch, the development of a prototype air-quality monitoring station based on the ESP32 microcontroller (ESP32-WROOM-32 module) and four low-cost sensors: a PMS7003 laser-scattering particulate matter sensor (PM1.0, PM2.5 and PM10), a BME280 temperature, humidity and pressure sensor, a DHT22 temperature and humidity sensor, and an MQ135 metal-oxide gas sensor. It covers the theory of the measured phenomena, the interface circuits and the communication protocols, the incremental ten-stage assembly method, the software environment, the libraries, every test program and the final firmware, which reads all sensors without blocking, validates the data, serves its own web page over a Wi-Fi access point and uploads the measurements to ThingSpeak. Laboratory results show all four sensors working together without communication errors, BME280/DHT22 agreement within factory tolerances, PM2.5 around 94 µg/m³ in the laboratory and a +44 % MQ135 response to alcohol vapour.

Keywords: *air quality; ESP32; particulate matter; low-cost sensors; Internet of Things.*

Sumário

1 Introdução 7
1.1 Contexto e motivação 7
1.2 Objetivos 7
1.3 Escopo 7
1.4 Organização do documento 8

2 Fundamentação teórica 9
2.1 Qualidade do ar e seus indicadores 9
2.1.1 Material particulado 9
2.1.2 Gases 10
2.1.3 Variáveis meteorológicas 10
2.2 Fundamentos de eletrônica aplicados ao projeto 10
2.2.1 Tensão, corrente, resistência e potência 10
2.2.2 Divisor de tensão 10
2.2.3 Níveis lógicos, entradas de dreno aberto e resistores de pull-up 11
2.2.4 Terra comum e alimentação 11
2.2.5 Conversão analógico-digital 11
2.3 O microcontrolador ESP32 12
2.4 Protocolos de comunicação 13
2.4.1 UART (comunicação serial assíncrona) 13
2.4.2 I²C (Inter-Integrated Circuit) 14
2.4.3 Protocolo de um fio do DHT22 14
2.5 Os sensores 15
2.5.1 DHT22 (AM2302): temperatura e umidade 15
2.5.2 BME280: pressão, temperatura e umidade 15
2.5.3 PMS7003: material particulado por espalhamento de laser 16
2.5.4 MQ135: gases por semicondutor de óxido metálico 18

3 Materiais e métodos 21
3.1 Materiais 21
3.2 Ferramentas e ambiente 22
3.3 Metodologia de montagem incremental 22
3.4 Preparação do ambiente de software, do zero 23
3.4.1 Instalação da Arduino IDE 23
3.4.2 Pacote de placas ESP32 23
3.4.3 Acesso à porta USB no Linux 24
3.4.4 Bibliotecas 24
3.4.5 Rotina de gravação e teste 25
3.5 Mapa de ligações 25

4 Desenvolvimento por etapas 28
4.1 Etapa 3: o ESP32 sozinho 28
4.2 Etapa 4: DHT22 29
4.3 Etapa 5: sensor de pressão no barramento I²C 30

4.4 Etapa 6: PMS7003 32
4.5 Etapa 7: MQ135 com divisor de tensão 34
4.6 Etapa 8: estação integrada 35
4.7 Etapa 9: versão final com página Wi-Fi e ThingSpeak 37
4.7.1 Configuração do ThingSpeak 38
4.8 Etapa 10: alimentação sem computador 39

5 Arquitetura do firmware final 41
5.1 Organização em blocos 42
5.2 Estruturas de dados 42
5.3 Regras de validação 43
5.4 Comunicação com a nuvem e robustez da rede 43

6 Resultados e discussão 44
6.1 Síntese dos resultados 44
6.2 Discussão 44
6.2.1 Concordância entre os sensores de temperatura e umidade 44
6.2.2 Material particulado no laboratório 44
6.2.3 Sensor de gases 45
6.2.4 Robustez do sistema 45

7 Dificuldades encontradas e soluções 46

8 Limitações e trabalhos futuros 48

9 Conclusão 49

Referências 50

Apêndice A – Códigos completos 51

Apêndice B – Comandos de terminal utilizados 71

Apêndice C – Lista de verificações elétricas 72

Lista de figuras

Figura 1 – Diagrama de blocos da estação: sensores, interfaces, processamento no ESP32 e saídas de dados. 8
Figura 2 – Linha de dados do DHT22: dois dispositivos de dreno aberto e um resistor de pull-up de 4,7 kΩ. 11
Figura 3 – Barramento I²C (SDA e SCL) com os resistores de pull-up, normalmente já presentes no módulo BME280. 11
Figura 4 – Simulação de Monte Carlo (4 000 repetições): leitura de 0,5 V com ruído gaussiano hipotético de 20 mV. A média de 20 amostras (Etapa 7) e a de 50 amostras (programa final) reduzem a dispersão por √N. 12
Figura 5 – Simulação da forma de onda UART 8N1 a 9600 bps dos dois primeiros bytes de todo pacote do PMS7003: 0x42 ('B') e 0x4D ('M'). 13
Figura 6 – Simulação da transação I²C a 100 kHz que lê o registrador de identificação (0xD0) do sensor no endereço 0x76. A resposta 0x60 identifica um BME280; um BMP280 responderia 0x58. 14
Figura 7 – Simulação do quadro completo do DHT22 para 50,4 % e 25,6 °C (valores reais da Etapa 4), com ampliação da resposta e do primeiro byte. 15
Figura 8 – Pressão em função da altitude na atmosfera padrão, com a faixa medida no laboratório (1013,6 a 1014,0 hPa). 16
Figura 9 – Representação esquemática simplificada do princípio de medição do PMS7003. 17
Figura 10 – Modelo de barreira entre grãos do SnO_2: em ar limpo (esquerda) e com gás redutor (direita). Representação qualitativa. 18
Figura 11 – Circuito equivalente do módulo MQ135 (aquecedor, R_S e R_L) e do divisor de proteção ligado ao GPIO34. 19
Figura 12 – (a) Tensão no GPIO34 em função da saída AO, com e sem divisor. (b) Simulação do efeito de carga do divisor sobre AO para dois valores possíveis de R_L. 19
Figura 13 – (a) Curva característica pelo modelo da Equação 6. (b) Conversão das tensões reais da Etapa 7 em R_S, supondo R_L = 1 kΩ (hipótese a confirmar na placa). 20
Figura 14 – Placa adaptadora do PMS7003 usada no protótipo. Foi usado o conector P3, de 4 pinos com passo de 2,54 mm. 22
Figura 15 – Esquema de ligações completo do protótipo. Os trilhos inferiores representam as linhas de 3,3 V, 5 V e GND da protoboard. 26
Figura 16 – Resultado real da Etapa 3 no Monitor Serial: ESP32-D0WD-V3, revisão 3, 2 núcleos, 240 MHz e 4 MB de flash. 29
Figura 17 – Resultado real da Etapa 4. Com o sensor fechado na mão, temperatura e umidade sobem. 30
Figura 18 – Dados reais da Figura 17 em gráfico (uma leitura a cada 2,5 s). 30
Figura 19 – Resultado real da Etapa 5: dispositivo em 0x76 com ID 0x60, ou seja, um BME280. 31
Figura 20 – Dados reais da Etapa 5b após estabilização: BME280 e DHT22 lado a lado (leituras a cada 2,5 s). 31
Figura 21 – Resultado real da Etapa 6: mais de 100 pacotes válidos e nenhum pacote ruim. 33
Figura 22 – Dados reais da Figura 21 em gráfico, comparados com a diretriz da OMS para média de 24 h. 33

Figura 23 – Resultado real da Etapa 7 durante o teste do álcool (após 6 minutos de aquecimento). 35
Figura 24 – Dados reais do MQ135: (a) início, com o sensor ainda frio; (b) teste com álcool em gel aproximado a cerca de 5 cm. 35
Figura 25 – Resultado real da Etapa 8, incluindo o teste de robustez: nas leituras #13 e #14 o fio TX do PMS7003 foi desconectado, o sensor aparece como FALHA e os demais continuam funcionando. 36
Figura 26 – Página web do firmware final (tela de celular, dividida em duas colunas). Simulação: o HTML é exatamente o gravado no ESP32, alimentado com dados fictícios próximos aos medidos. 38
Figura 27 – Orçamento de corrente estimado do protótipo, a partir dos datasheets e de valores típicos de operação do ESP32 com Wi-Fi. 40
Figura 28 – Fluxograma do firmware final. 41
Figura 29 – Escalonamento cooperativo: instantes em que cada tarefa é executada durante o primeiro minuto de funcionamento. 42
Figura 30 – Erro real de compilação causado pela incompatibilidade entre a IDE 1.8.19 em snap e o pacote esp32 3.3.12. A compilação do código em si funcionou ("Sketch uses 273192 bytes"). 47

Lista de tabelas

Tabela 1 – Diretrizes da OMS (2021) para material particulado. 9
Tabela 2 – Faixas usadas pelo firmware para classificar o PM2,5 (referência: média de 24 h). 9
Tabela 3 – Restrições dos pinos do ESP32 consideradas na escolha do mapa de ligações. 13
Tabela 4 – Principais características do DHT22. 15
Tabela 5 – Principais características do BME280. 16
Tabela 6 – Principais características do PMS7003. 17
Tabela 7 – Estrutura do pacote de dados do PMS7003 (modo ativo). 18
Tabela 8 – Principais características do MQ135. 20
Tabela 9 – Lista de materiais do protótipo. 21
Tabela 10 – Etapas do desenvolvimento e critérios de aprovação. 23
Tabela 11 – Configuração da placa usada na gravação (menu Tools). 24
Tabela 12 – Bibliotecas utilizadas. 25
Tabela 13 – Tabela de ligações. 26
Tabela 14 – Simulação: pacote de 32 bytes montado com os valores reais da primeira leitura da Etapa 6 (PM1,0 = 62, PM2,5 = 94, PM10 = 98 µg/m³; 5442 partículas >0,3 µm; 20 >2,5 µm), em hexadecimal. 32
Tabela 15 – Rotas do servidor web do ESP32. 37
Tabela 16 – Campos do canal ThingSpeak. 39
Tabela 17 – Blocos do programa final e suas funções. 42
Tabela 18 – Critérios usados pelo firmware para considerar cada leitura válida. 43
Tabela 19 – Valores medidos no laboratório durante as etapas. 44
Tabela 20 – Dificuldades encontradas, causas e soluções. 46
Tabela 21 – Limitações atuais e próximos passos do protótipo. 48
Tabela 22 – Verificações com o multímetro antes de cada energização. 72

1 Introdução

1.1 Contexto e motivação

A poluição do ar é um dos principais fatores ambientais de risco à saúde. Partículas finas em suspensão, com diâmetro aerodinâmico de até 2,5 µm (PM2,5), penetram profundamente no sistema respiratório e estão associadas a doenças cardiovasculares e respiratórias. Por esse motivo, a Organização Mundial da Saúde (OMS) revisou em 2021 suas diretrizes, tornando-as mais restritivas (WHO, 2021). Estações oficiais de monitoramento são caras e esparsas; em contrapartida, a popularização de microcontroladores com conectividade sem fio e de sensores ópticos e eletroquímicos de baixo custo tornou possível construir estações compactas, capazes de medir com resolução espacial e temporal muito maior, ainda que com exatidão inferior à dos instrumentos de referência.

Este documento registra integralmente o desenvolvimento de um protótipo funcional desse tipo de estação, com o objetivo de servir tanto como relatório do que foi feito quanto como guia para que qualquer pessoa reproduza exatamente a mesma estação a partir do zero.

1.2 Objetivos

Objetivo geral: projetar, montar, programar e validar o protótipo de uma estação de monitoramento da qualidade do ar baseada no ESP32, capaz de medir material particulado, gases, temperatura, umidade e pressão, e de disponibilizar os dados localmente e na nuvem.

Objetivos específicos:

- explicar o princípio físico de funcionamento de cada sensor e dos circuitos de interface;
- definir um mapa de ligações seguro, compatível com os níveis de tensão do ESP32;
- preparar o ambiente de desenvolvimento em Linux e registrar os problemas encontrados e suas soluções;
- integrar e testar os sensores um por vez, com critérios objetivos de aprovação em cada etapa;
- desenvolver um firmware final não bloqueante, tolerante a falhas, com página web própria e envio ao ThingSpeak;
- analisar os resultados obtidos e discutir as limitações dos sensores de baixo custo.

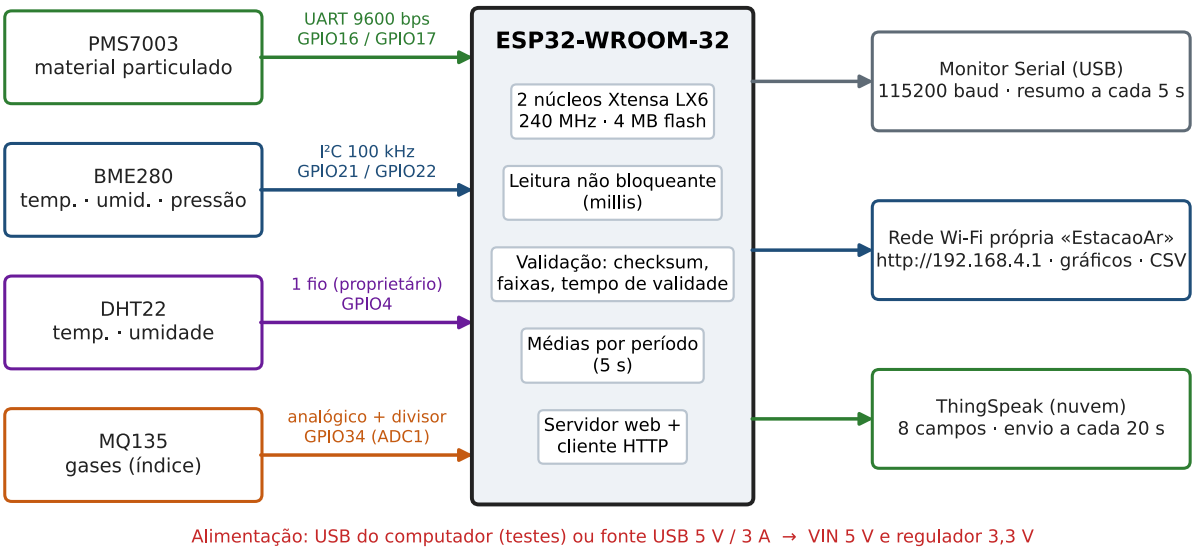
1.3 Escopo

O escopo deste documento é exclusivamente o **protótipo**: montagem em protoboard, alimentação por USB e o firmware completo. O projeto da estrutura física definitiva (caixa, abrigo de radiação, placa soldada e instalação externa) foi iniciado em paralelo, mas está **fora do escopo** deste texto. Quando um assunto depende da estrutura final, ele é apenas citado como trabalho futuro.

1.4 Organização do documento

O Capítulo 2 apresenta a fundamentação teórica: indicadores de qualidade do ar, fundamentos de eletrônica, o microcontrolador ESP32, os protocolos de comunicação e o funcionamento de cada sensor. O Capítulo 3 descreve materiais, ferramentas, a metodologia de montagem incremental, a instalação do ambiente de software e o mapa de ligações. O Capítulo 4 detalha cada etapa de montagem, com os programas e os resultados reais. O Capítulo 5 explica a arquitetura do firmware final. O Capítulo 6 consolida e discute os resultados. O Capítulo 7 reúne as dificuldades encontradas e as soluções adotadas, e o Capítulo 8 trata das limitações e dos próximos passos. Os Apêndices trazem os códigos completos, os comandos utilizados e a lista de verificações elétricas.

Figura 1 – Diagrama de blocos da estação: sensores, interfaces, processamento no ESP32 e saídas de dados.



Fonte: Elaborado pelos autores.

2 Fundamentação teórica

2.1 Qualidade do ar e seus indicadores

2.1.1 Material particulado

Material particulado (MP, ou PM, do inglês *particulate matter*) é o conjunto de partículas sólidas e líquidas suspensas no ar. Ele é classificado pelo **diâmetro aerodinâmico**: PM10 reúne as partículas com até 10 µm, PM2,5 as partículas com até 2,5 µm e PM1,0 as partículas com até 1 µm. Como referência, um fio de cabelo humano tem de 50 a 70 µm de diâmetro. As partículas grossas (entre 2,5 e 10 µm) vêm principalmente de poeira, ressuspensão do solo e pólen; as finas (abaixo de 2,5 µm) vêm sobretudo de processos de combustão (veículos, queimadas, indústria) e de reações químicas na atmosfera. A concentração é expressa em massa por volume, em **µg/m³** (microgramas por metro cúbico).

Quanto menor a partícula, mais profundamente ela penetra no sistema respiratório: PM10 alcança a traqueia e os brônquios, enquanto PM2,5 chega aos alvéolos pulmonares e PM1,0 pode atingir a corrente sanguínea. Por isso o PM2,5 é o indicador mais usado em índices de qualidade do ar.

Tabela 1 – Diretrizes da OMS (2021) para material particulado.

Poluente	Média de 24 horas	Média anual
PM2,5	15 µg/m³	5 µg/m³
PM10	45 µg/m³	15 µg/m³

Fonte: WHO (2021).

No Brasil, a Resolução CONAMA nº 491/2018 estabelece padrões de qualidade do ar, e os órgãos ambientais divulgam um índice de qualidade do ar (IQAR) com cinco faixas qualitativas. O firmware deste protótipo usa as faixas do índice para o PM2,5 apresentadas na Tabela 2 apenas como indicação visual da leitura do momento. Elas foram definidas para médias de 24 horas, e uma leitura instantânea não substitui essa média.

Tabela 2 – Faixas usadas pelo firmware para classificar o PM2,5 (referência: média de 24 h).

Classificação	PM2,5 (µg/m³)	Cor usada na página web
BOA	0 a 25	verde
MODERADA	> 25 a 50	amarelo
RUIM	> 50 a 75	laranja
MUITO RUIM	> 75 a 125	vermelho
PÉSSIMA	> 125	roxo

Fonte: Faixas do IQAR para MP2,5 adotadas por órgãos ambientais brasileiros (CETESB; CONAMA, 2018); implementação na função `classificarPm25()`.

2.1.2 Gases

Além das partículas, a qualidade do ar é afetada por gases como monóxido de carbono (CO), óxidos de nitrogênio (NOx), amônia (NH_3), compostos orgânicos voláteis (COV, como solventes e álcool) e dióxido de carbono (CO_2). Em ambientes internos, o aumento de CO_2 e de COV indica ventilação insuficiente. Sensores de baixo custo do tipo semicondutor, como o MQ135, respondem a vários desses gases ao mesmo tempo e, por isso, fornecem um **índice relativo** de contaminação, e não a concentração de um gás específico (seção 2.5.4).

2.1.3 Variáveis meteorológicas

Temperatura, umidade relativa e pressão atmosférica não são poluentes, mas são indispensáveis para interpretar os demais dados. A umidade relativa elevada (acima de cerca de 70 a 80 %) faz partículas higroscópicas absorverem água e "incharem", o que leva sensores ópticos a superestimar a massa de PM. A resistência dos sensores de gás também varia com temperatura e umidade. A pressão indica a altitude e a situação meteorológica: pressão alta e estável costuma estar associada a ar parado, condição que favorece o acúmulo de poluentes.

2.2 Fundamentos de eletrônica aplicados ao projeto

2.2.1 Tensão, corrente, resistência e potência

Todo o circuito da estação é de corrente contínua. A relação entre tensão V (volts), corrente I (ampères) e resistência R (ohms) é dada pela **lei de Ohm**, e a potência dissipada é o produto de tensão e corrente:

$$V = R \cdot I \qquad P = V \cdot I = R \cdot I^2 \tag{1}$$

Um exemplo do próprio projeto: o aquecedor do MQ135 tem resistência de cerca de 33 Ω e é alimentado com 5 V. A corrente é 5 / 33 ≈ 0,15 A e a potência é 5 × 0,15 ≈ 0,76 W. É por isso que o sensor fica quente ao toque e é o maior consumidor de energia entre os sensores.

2.2.2 Divisor de tensão

Dois resistores em série entre uma tensão de entrada e o terra formam um divisor de tensão. A tensão no ponto médio é uma fração fixa da entrada:

$$V_{\text{saída}} = V_{\text{entrada}} \cdot \frac{R_2}{R_1 + R_2} \tag{2}$$

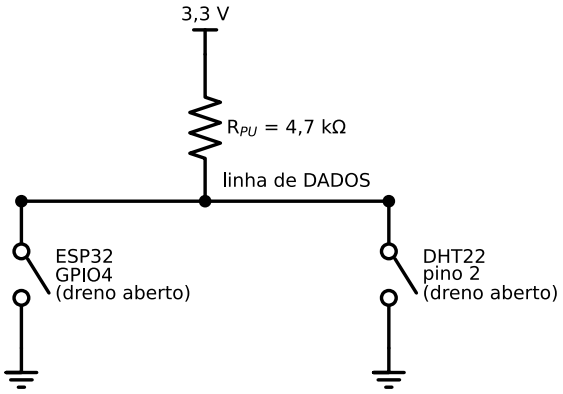
Com $R_1 = R_2 = 4{,}7$ kΩ, a fração é exatamente 1/2. Assim, se a saída analógica do MQ135 chegar a 5 V, o pino do ESP32 recebe 2,5 V, abaixo do limite de 3,3 V. A resistência equivalente de saída do divisor (o equivalente de Thévenin) é $R_1 \parallel R_2 = 2{,}35$ kΩ, baixa o suficiente para o conversor analógico-digital do ESP32. A carga que o divisor impõe à saída do módulo (9,4 kΩ ao terra) altera um pouco a tensão em AO; esse efeito é simulado na seção 2.5.4 e é absorvido pela calibração.

2.2.3 Níveis lógicos, entradas de dreno aberto e resistores de pull-up

O ESP32 trabalha com lógica de **3,3 V**: um sinal digital é interpretado como nível alto quando está acima de cerca de 0,75·VDD (≈ 2,5 V) e como nível baixo abaixo de cerca de 0,25·VDD (≈ 0,8 V). Os pinos **não toleram 5 V**. Por isso, o BME280 e o DHT22 foram alimentados com 3,3 V, o que garante que seus sinais de dados também sejam de 3,3 V.

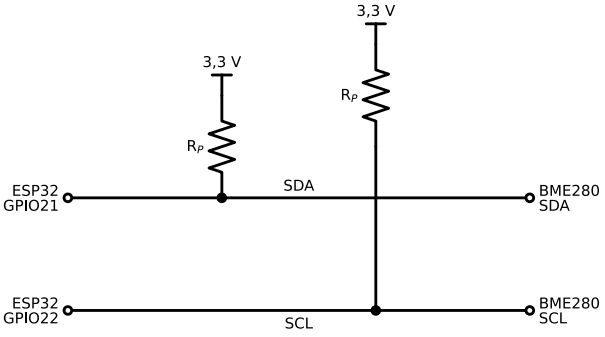
Nos barramentos compartilhados (o protocolo do DHT22 e o I²C do BME280), cada dispositivo só consegue **puxar a linha para 0 V** (saída de dreno aberto); nenhum deles força o nível alto. Quem mantém a linha em nível alto, quando todos estão "soltos", é um **resistor de pull-up** ligado ao 3,3 V. Isso evita curto-circuito quando dois dispositivos tentam falar ao mesmo tempo e permite a comunicação nos dois sentidos pelo mesmo fio. Sem o resistor, a linha fica flutuando e a comunicação falha. Com 4,7 kΩ, a corrente quando a linha está em 0 V é de apenas 3,3 / 4700 ≈ 0,7 mA.

Figura 2 – Linha de dados do DHT22: dois dispositivos de dreno aberto e um resistor de pull-up de 4,7 kΩ.



Fonte: Elaborado pelos autores.

Figura 3 – Barramento I²C (SDA e SCL) com os resistores de pull-up, normalmente já presentes no módulo BME280.



Fonte: Elaborado pelos autores.

2.2.4 Terra comum e alimentação

Tensão é sempre uma diferença entre dois pontos. Para que o ESP32 e os sensores concordem sobre o que é "0 V", todos os terminais GND precisam estar interligados (**terra comum**). Sem isso, os sinais perdem a referência e a leitura vira ruído. A placa DevKit recebe 5 V pelo USB (ou pelo pino VIN), entrega essa tensão aos sensores de 5 V pelo próprio pino VIN e gera 3,3 V com um regulador linear (tipicamente um AMS1117-3.3), disponível no pino 3V3. Duas fontes de 5 V nunca devem ser ligadas ao mesmo tempo (USB e fonte no VIN), pois a pequena diferença entre elas faz circular corrente de uma para a outra.

2.2.5 Conversão analógico-digital

O conversor analógico-digital (ADC) do ESP32 é do tipo aproximações sucessivas, com **12 bits** de resolução: a tensão de entrada é convertida em um número inteiro de 0 a $2^{12} - 1 = 4095$. Para medir tensões maiores que a referência interna (~1,1 V), a entrada passa por um **atenuador**;

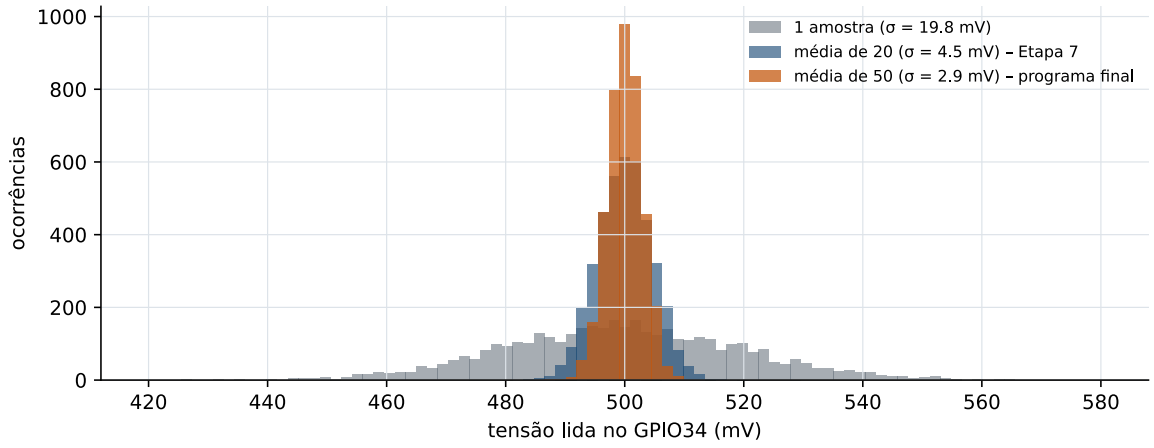
com atenuação de 11 dB, a faixa recomendada pela fabricante vai de cerca de 0,15 V a 2,45 V. O passo de quantização (LSB) é da ordem de 0,8 mV.

O ADC do ESP32 não é perfeitamente linear e cada chip tem pequenas diferenças. Por isso, o firmware usa a função `analogReadMilliVolts()`, que aplica a calibração gravada de fábrica no chip (eFuse) e devolve a tensão diretamente em milivolts. Isso aparece nos dados reais da Etapa 7: uma leitura bruta de 445 corresponde a 501 mV, e não aos 359 mV que uma conversão linear ingênua (445/4095 × 3,3 V) daria.

O ruído de cada leitura individual é reduzido calculando a **média** de *N* amostras. Para ruído aleatório independente, o desvio padrão da média cai com a raiz quadrada do número de amostras:

$$\sigma_{\text{média}} = \frac{\sigma}{\sqrt{N}} \tag{3}$$

Figura 4 – Simulação de Monte Carlo (4 000 repetições): leitura de 0,5 V com ruído gaussiano hipotético de 20 mV. A média de 20 amostras (Etapa 7) e a de 50 amostras (programa final) reduzem a dispersão por √N.



Fonte: Elaborado pelos autores.

2.3 O microcontrolador ESP32

O ESP32, da Espressif Systems, é um sistema em um chip (SoC) com dois núcleos de processamento Xtensa LX6 de 32 bits de até 240 MHz, 520 KB de SRAM, Wi-Fi 802.11 b/g/n em 2,4 GHz e Bluetooth. O módulo **ESP32-WROOM-32** usado no projeto integra o chip (ESP32-D0WD-V3, revisão 3), 4 MB de memória flash, cristal e antena impressa. A placa de desenvolvimento **DevKit de 30 pinos** acrescenta o regulador de 3,3 V, o conector micro-USB, o conversor USB-serial, os botões EN (reinício) e BOOT (modo de gravação), um LED de alimentação e um LED azul no GPIO2.

Na placa usada, o conversor USB-serial é um **QinHeng CH343/CH9102** (identificação USB 1a86:55d4). Diferentemente dos mais antigos CP2102 e CH340, ele se apresenta ao Linux como dispositivo CDC-ACM e aparece como `/dev/ttyACM0`, e não como `/dev/ttyUSB0`. Esse detalhe foi decisivo na Etapa 3 (Capítulo 7).

Tabela 3 – Restrições dos pinos do ESP32 consideradas na escolha do mapa de ligações.

Pinos	Característica	Consequência no projeto
GPIO0, 2, 5, 12, 15	*Strapping pins*: seu nível no momento do reset define o modo de inicialização	Evitados para sensores (exceto o LED interno no GPIO2, usado só no teste)
GPIO6 a 11	Ligados à memória flash interna	Não disponíveis (nem aparecem na placa de 30 pinos)
GPIO1 e 3 (TX0/RX0)	UART0, usada pelo USB para gravação e Monitor Serial	Reservados
GPIO34 a 39	Somente entrada, sem pull-up interno, pertencem ao ADC1	GPIO34 escolhido para o MQ135
ADC2 (GPIO 0, 2, 4, 12–15, 25–27)	Indisponível para leitura analógica com o Wi-Fi ligado	MQ135 no ADC1 para conviver com o Wi-Fi
GPIO16 e 17	Pinos padrão da UART2	PMS7003
GPIO21 e 22	Pinos padrão do I²C no Arduino-ESP32	BME280

Fonte: Espressif (2023a; 2023b); adaptado pelos autores.

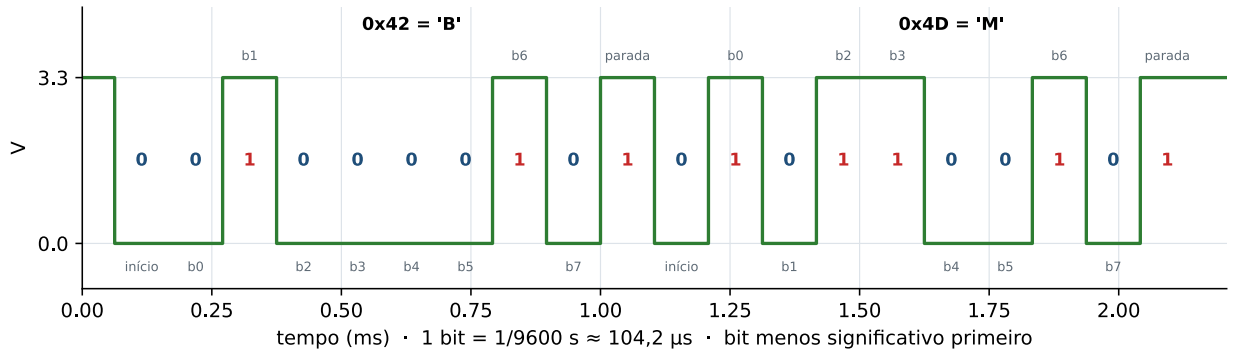
O ESP32 pode operar o Wi-Fi como **estação** (STA, conectando-se a um roteador), como **ponto de acesso** (AP, criando a própria rede) ou nos dois modos simultaneamente (AP+STA). O firmware final usa o modo AP+STA: cria a rede "EstacaoAr" para a página local e, se configurado, conecta-se a uma rede com internet para enviar dados à nuvem.

2.4 Protocolos de comunicação

2.4.1 UART (comunicação serial assíncrona)

Na UART, os bytes são enviados um bit por vez, por um único fio em cada sentido (TX transmite, RX recebe), sem sinal de relógio compartilhado. Os dois lados combinam previamente a velocidade (taxa em bits por segundo, ou *baud*) e o formato do quadro. O PMS7003 usa **9600 bps no formato 8N1**: um bit de início (0), oito bits de dados enviados do menos significativo para o mais significativo, nenhum bit de paridade e um bit de parada (1). Cada bit dura 1/9600 s ≈ 104,2 µs, e cada byte ocupa 10 bits, ou seja, cerca de 1,04 ms. Como um transmite no fio em que o outro escuta, as ligações são **cruzadas**: TX do sensor no RX do ESP32 e vice-versa.

Figura 5 – Simulação da forma de onda UART 8N1 a 9600 bps dos dois primeiros bytes de todo pacote do PMS7003: 0x42 ('B') e 0x4D ('M').

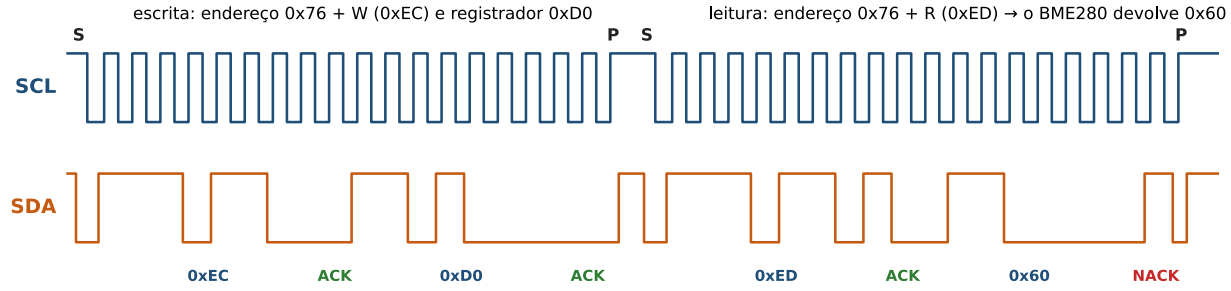


Fonte: Elaborado pelos autores.

2.4.2 I²C (Inter-Integrated Circuit)

O I²C é um barramento síncrono de dois fios: **SCL**, o relógio gerado pelo mestre (o ESP32), e **SDA**, os dados, que trafegam nos dois sentidos. Vários dispositivos podem compartilhar os mesmos fios porque cada um tem um **endereço** de 7 bits. Uma transação começa com a condição de início (S: SDA desce enquanto SCL está alto) e termina com a de parada (P: SDA sobe com SCL alto). Depois de cada byte, o receptor confirma com um bit ACK (0). O BME280 responde no endereço 0x76 ou 0x77, dependendo do nível do pino SDO no módulo. Para ler um registrador, o mestre primeiro escreve o número do registrador e depois faz uma leitura. A Figura 6 mostra exatamente a leitura feita pelo programa da Etapa 5: o registrador de identificação 0xD0, que devolveu 0x60.

Figura 6 – Simulação da transação I²C a 100 kHz que lê o registrador de identificação (0xD0) do sensor no endereço 0x76. A resposta 0x60 identifica um BME280; um BMP280 responderia 0x58.



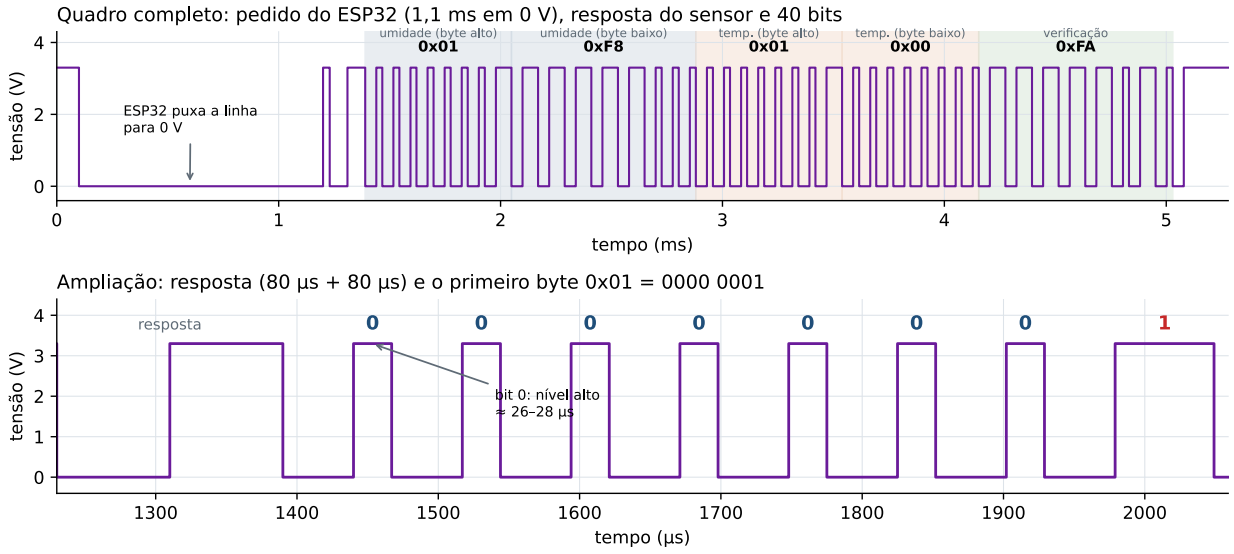
Fonte: Elaborado pelos autores.

2.4.3 Protocolo de um fio do DHT22

O DHT22 usa um protocolo proprietário em um único fio, em que **a informação está na duração dos pulsos**. O ESP32 inicia a conversa mantendo a linha em 0 V por pelo menos 1 ms (a biblioteca usa 1,1 ms) e depois a solta. O sensor responde com 80 µs em nível baixo e 80 µs em nível alto e, em seguida, envia 40 bits. Todo bit começa com 50 µs em nível baixo; o que vem depois decide o valor: cerca de 26 a 28 µs em nível alto representam 0, e cerca de 70 µs representam 1.

Os 40 bits formam cinco bytes: dois de umidade (em décimos de %), dois de temperatura (em décimos de °C, com o bit mais significativo indicando temperatura negativa) e um byte de verificação, igual aos 8 bits menos significativos da soma dos quatro anteriores. Para a leitura real de 50,4 % e 25,6 °C da Etapa 4: umidade = 504 = 0x01F8, temperatura = 256 = 0x0100 e verificação = (0x01 + 0xF8 + 0x01 + 0x00) mod 256 = 0xFA.

Figura 7 – Simulação do quadro completo do DHT22 para 50,4 % e 25,6 °C (valores reais da Etapa 4), com ampliação da resposta e do primeiro byte.



Fonte: Elaborado pelos autores.

2.5 Os sensores

2.5.1 DHT22 (AM2302): temperatura e umidade

O DHT22 contém um **sensor capacitivo de umidade**, formado por um dielétrico polimérico que absorve vapor d'água: a permissividade do material, e portanto a capacitância, aumenta com a umidade. A temperatura é medida por um **termistor** (NTC), cuja resistência diminui com o aumento da temperatura. Um microcontrolador interno lê os dois elementos, aplica a calibração de fábrica e transmite o resultado pelo protocolo de um fio. O sensor usado é a versão solta de 4 pernas: 1 = VCC, 2 = dados, 3 = não conectada e 4 = GND.

Tabela 4 – Principais características do DHT22.

Parâmetro	Valor
Alimentação	3,3 a 6 V (usado: 3,3 V)
Faixa de umidade / exatidão	0 a 100 % UR / ±2 % UR (máx. ±5 %)
Faixa de temperatura / exatidão	−40 a 80 °C / ±0,5 °C
Resolução	0,1 % UR e 0,1 °C
Intervalo mínimo entre leituras	2 s
Interface	1 fio proprietário, com pull-up externo

Fonte: Aosong (s.d.).

2.5.2 BME280: pressão, temperatura e umidade

O BME280, da Bosch Sensortec, é um sensor MEMS (sistema microeletromecânico). A pressão é medida por uma **membrana de silício** com resistores piezorresistivos: a pressão do ar deforma a membrana, a deformação altera a resistência e o chip converte essa variação em número. Há também um sensor de temperatura, usado principalmente para compensar a medida de pressão, e

um sensor capacitivo de umidade. Cada chip sai de fábrica com **coeficientes de calibração** gravados em memória não volátil; a biblioteca lê esses coeficientes e aplica as equações de compensação do *datasheet*.

O BMP280 é externamente quase idêntico, mas não mede umidade. A diferença confiável entre os dois está no registrador de identificação 0xD0: **0x60 para o BME280** e **0x58 para o BMP280**. Na inspeção visual (Etapa 1) o sensor foi classificado como BMP280; a leitura do registrador na Etapa 5 mostrou que é um BME280 (seção 4.3).

Tabela 5 – Principais características do BME280.

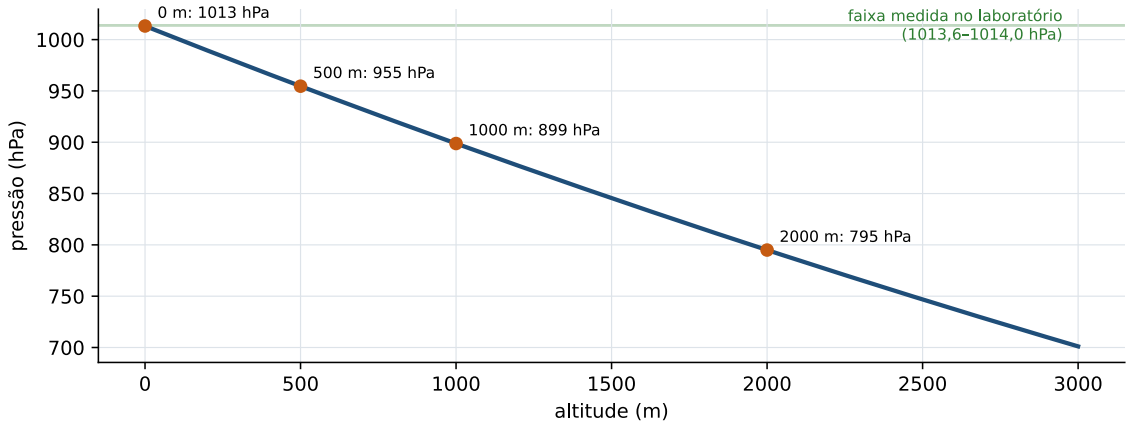
Parâmetro	Valor
Faixa de pressão	300 a 1100 hPa
Exatidão absoluta da pressão	±1 hPa
Exatidão relativa da pressão	±0,12 hPa (≈ ±1 m de altitude)
Exatidão de temperatura	±1 °C
Exatidão de umidade	±3 % UR
Endereço I²C	0x76 (SDO no GND) ou 0x77 (SDO no VDD)
Alimentação do módulo	pino VIN (com regulador); ligado em 3,3 V

Fonte: Bosch Sensortec (2022).

A pressão diminui com a altitude. Na atmosfera padrão internacional, a relação é aproximada por:

$$p(h) = 1013{,}25 \cdot (1 - 2{,}25577 \times 10^{-5} \cdot h)^{5{,}25588} \text{ hPa, com } h \text{ em metros} \quad (4)$$

Figura 8 – Pressão em função da altitude na atmosfera padrão, com a faixa medida no laboratório (1013,6 a 1014,0 hPa).



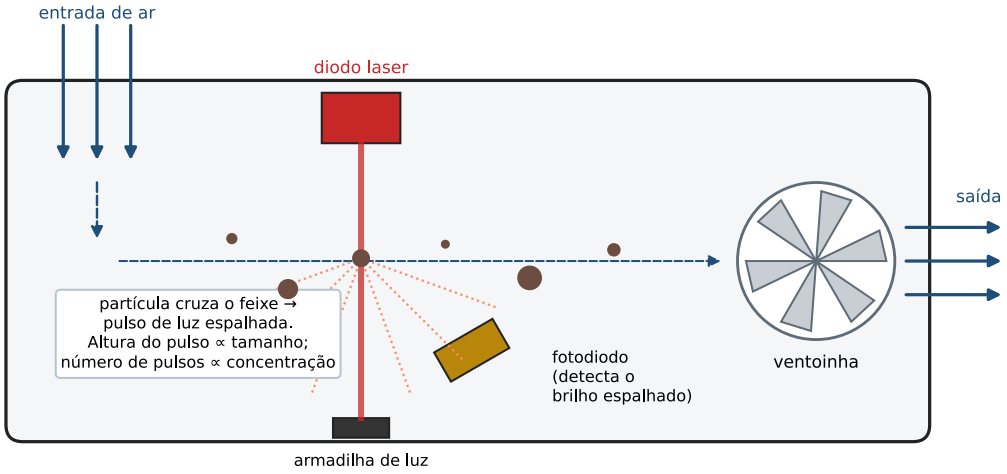
Fonte: Elaborado pelos autores.

2.5.3 PMS7003: material particulado por espalhamento de laser

O PMS7003, da Plantower, aspira o ar com uma pequena ventoinha para dentro de uma câmara escura atravessada por um feixe de **laser**. Cada partícula que cruza o feixe espalha uma pequena quantidade de luz (espalhamento de Mie, pois o tamanho das partículas é comparável ao

comprimento de onda), que é captada por um **fotodiodo**. O número de pulsos indica quantas partículas passaram, e a amplitude de cada pulso indica o tamanho. Um processador interno classifica as partículas em faixas de tamanho e estima a massa por volume com um modelo de densidade e forma, entregando PM1,0, PM2,5 e PM10 em µg/m³ e as contagens por faixa em 0,1 L de ar.

Figura 9 – Representação esquemática simplificada do princípio de medição do PMS7003.



Fonte: Elaborado pelos autores.

O sensor fornece dois conjuntos de valores de massa: "CF=1", calculado com o fator de calibração de partícula padrão de fábrica, e "ambiente atmosférico", indicado para medições do ar ambiente. O firmware usa este segundo conjunto. Como o método é óptico, partículas que absorvem água em umidade alta parecem maiores, o que tende a superestimar a massa.

Tabela 6 – Principais características do PMS7003.

Parâmetro	Valor
Faixas de tamanho	0,3–1,0 · 1,0–2,5 · 2,5–10 µm
Faixa efetiva / máxima (PM2,5)	0 a 500 µg/m³ / ≥ 1000 µg/m³
Resolução	1 µg/m³
Erro máximo de consistência	±10 µg/m³ (0 a 100) · ±10 % (100 a 500 µg/m³)
Tempo de resposta total	≤ 10 s
Alimentação / corrente ativa	4,5 a 5,5 V / ≤ 100 mA
Nível lógico dos dados	3,3 V (compatível direto com o ESP32)
Interface	UART 9600 bps, 8N1, pacote de 32 bytes

Fonte: Plantower (2016).

Cada pacote tem 32 bytes, com todos os valores de 16 bits enviados com o byte mais significativo primeiro. A soma de verificação é a soma simples de todos os bytes de 0 a 29, comparada com o valor de 16 bits dos bytes 30 e 31.

Tabela 7 – Estrutura do pacote de dados do PMS7003 (modo ativo).

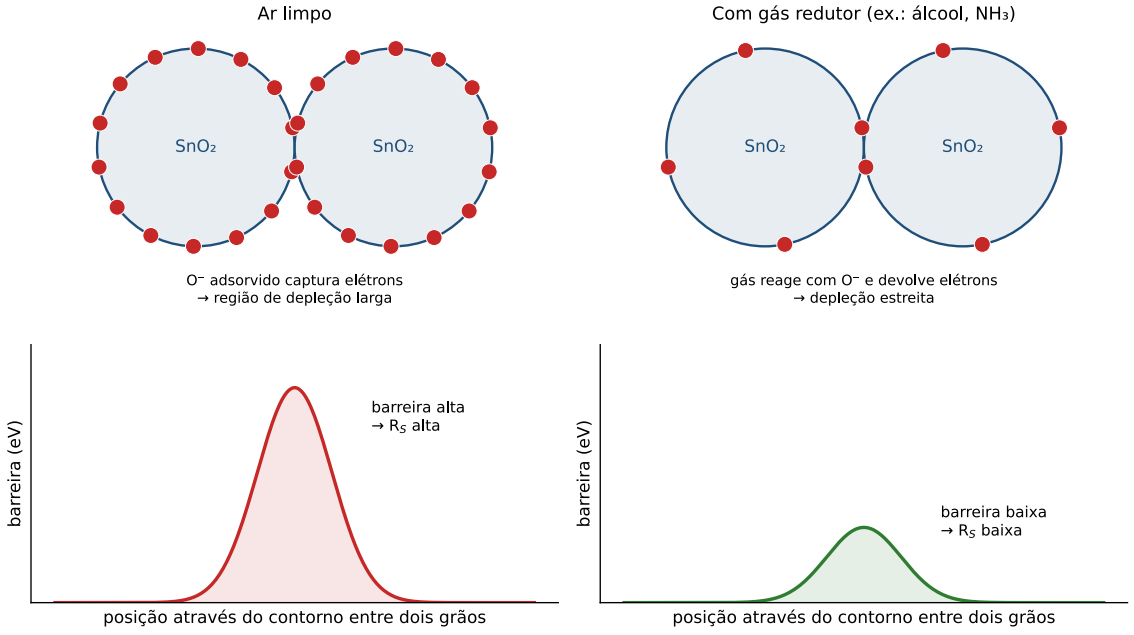
Bytes	Conteúdo	Uso no firmware
0–1	Início fixo 0x42 0x4D ('B','M')	sincronização do pacote
2–3	Comprimento do quadro = 28	validação
4–9	PM1,0, PM2,5 e PM10 (CF=1)	não usados
10–15	PM1,0, PM2,5 e PM10 (ambiente atmosférico)	**valores exibidos e enviados**
16–27	Contagens >0,3, >0,5, >1,0, >2,5, >5 e >10 µm em 0,1 L	>0,3 e >2,5 µm exibidos na Etapa 6
28–29	Reservado (versão e código de erro)	não usados
30–31	Soma de verificação	validação

Fonte: Plantower (2016).

2.5.4 MQ135: gases por semicondutor de óxido metálico

O elemento sensível do MQ135 é uma camada de **dióxido de estanho (SnO_2)**, um semicondutor do tipo n, mantida aquecida por um filamento interno. Em ar limpo, moléculas de oxigênio se adsorvem na superfície dos grãos de SnO_2 e capturam elétrons do material, criando uma região empobrecida de portadores e uma **barreira de potencial** nas fronteiras entre grãos; a resistência R_S é alta. Na presença de gases redutores (álcool, amônia, fumaça, benzeno), esses gases reagem com o oxigênio adsorvido, devolvem elétrons ao semicondutor, a barreira diminui e a resistência cai. O aquecimento é necessário para que essas reações ocorram com velocidade adequada.

Figura 10 – Modelo de barreira entre grãos do SnO_2: em ar limpo (esquerda) e com gás redutor (direita). Representação qualitativa.



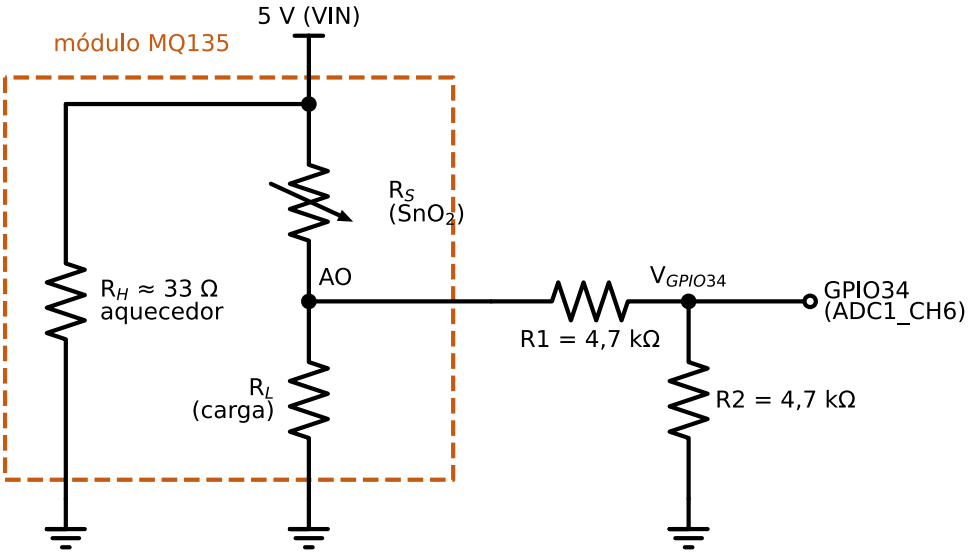
Fonte: Elaborado pelos autores.

No módulo comprado, o sensor forma um divisor com um resistor de carga R_L soldado na placa, e a saída AO é a tensão sobre R_L. Assim, quanto mais gás, menor R_S e **maior a tensão em AO**. Medindo V_{AO}, a resistência do sensor é calculada por:

$$R_S = R_L \cdot \frac{V_C - V_{AO}}{V_{AO}}\ , \text{com } V_C = 5\ \text{V} \tag{5}$$

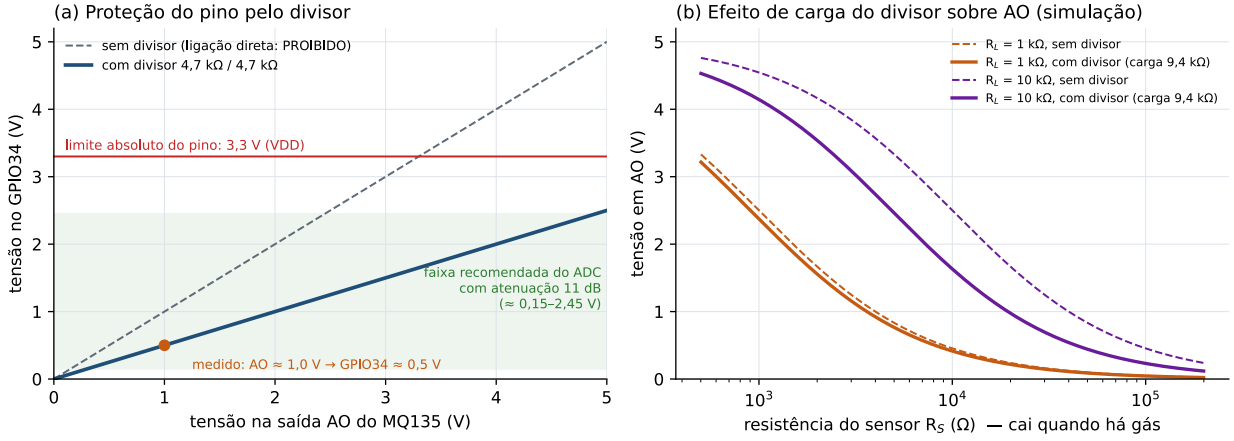
O valor de R_L varia entre fabricantes de módulos (é comum encontrar 1 kΩ, marcado "102", ou 10 kΩ, marcado "103") e precisa ser conferido na placa antes da calibração. Com o divisor de proteção ligado, R_L deve ser substituído pelo paralelo $R_L \| 9{,}4$ kΩ.

Figura 11 – Circuito equivalente do módulo MQ135 (aquecedor, R_S e R_L) e do divisor de proteção ligado ao GPIO34.



Fonte: Elaborado pelos autores.

Figura 12 – (a) Tensão no GPIO34 em função da saída AO, com e sem divisor. (b) Simulação do efeito de carga do divisor sobre AO para dois valores possíveis de R_L.

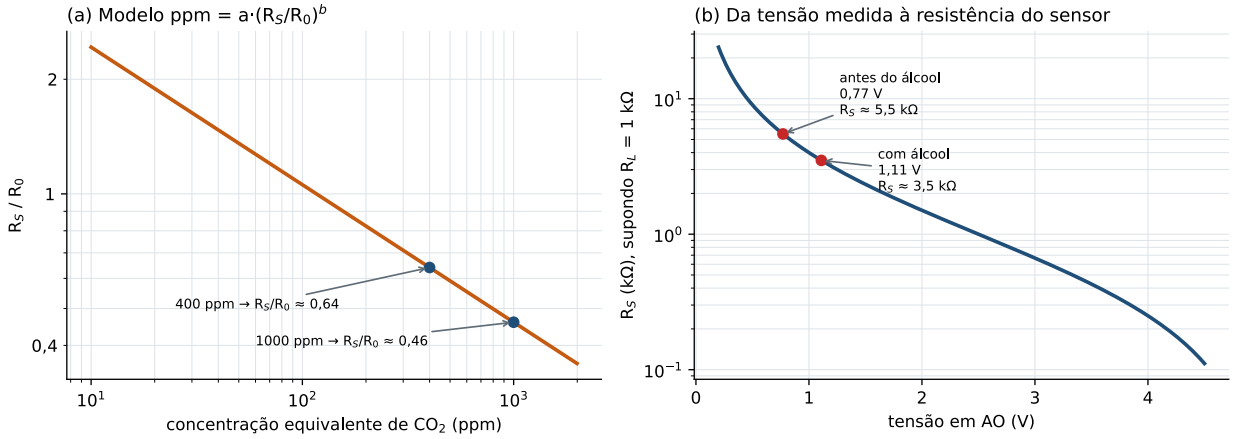


Fonte: Elaborado pelos autores.

A relação entre R_S e a concentração de gás, mostrada em escala log-log no *datasheet*, é aproximadamente uma reta, o que leva a um modelo de potência. A grandeza de referência é R_0, a resistência do sensor em uma condição conhecida (na prática, ar externo limpo, adotado como ~400 ppm de CO_2). Um ajuste muito usado para CO_2 equivalente é:

$$\text{ppm} = a \cdot (R_S/R_0)^b, \text{ com } a \approx 116{,}60 \text{ e } b \approx -2{,}769 \quad (6)$$

Figura 13 – (a) Curva característica pelo modelo da Equação 6. (b) Conversão das tensões reais da Etapa 7 em R_S, supondo R_L = 1 kΩ (hipótese a confirmar na placa).



Fonte: Elaborado pelos autores; coeficientes do ajuste da biblioteca MQ135 de G. Krocker, baseado na curva do datasheet (Hanwei, s.d.).

Limitação importante. O MQ135 não é seletivo: álcool, amônia, fumaça e CO_2 alteram a mesma resistência. A Equação 6 só produz "ppm de CO_2" se o único gás variando for o CO_2. Por isso, neste protótipo a saída do MQ135 é tratada como **índice relativo de contaminação**, em volts, até que a calibração de R_0 seja feita (Capítulo 8). O sensor novo também precisa de 24 a 48 horas ligado (queima inicial) para estabilizar.

Tabela 8 – Principais características do MQ135.

Parâmetro	Valor
Gases detectados	NH_3, NOx, álcool, benzeno, fumaça, CO_2 (não seletivo)
Tensão do aquecedor	5,0 ± 0,1 V
Resistência do aquecedor	33 Ω ± 5 %
Consumo do aquecedor	≤ 800 mW
Pré-aquecimento (queima inicial)	≥ 24 h na primeira utilização
Saídas do módulo	AO (analógica) e DO (comparador com ajuste por potenciômetro, não usada)

Fonte: Hanwei Electronics (s.d.).

3 Materiais e métodos

3.1 Materiais

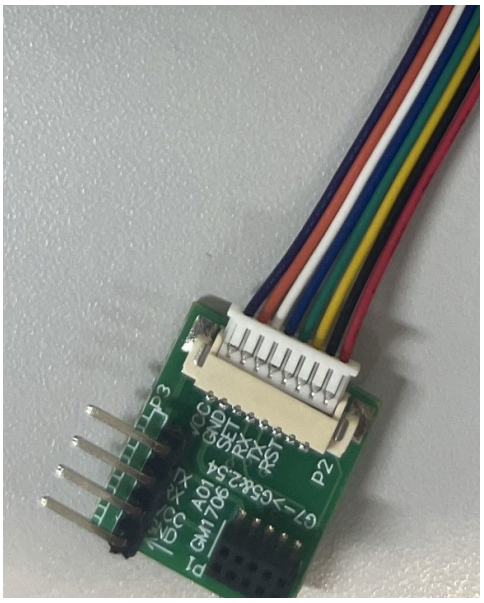
Tabela 9 – Lista de materiais do protótipo.

Item	Qtd.	Especificação usada	Função
Placa ESP32 DevKit	1	ESP32-WROOM-32, 30 pinos, micro-USB, USB-serial CH343/CH9102, 4 MB de flash	processamento, Wi-Fi
PMS7003	1	com placa adaptadora (conectores P1, P2 e P3) e cabo	material particulado
BME280	1	módulo de 4 pinos (VIN, GND, SCL, SDA), chip retangular	pressão, temperatura, umidade
DHT22 (AM2302)	1	sensor solto, 4 pernas, branco	temperatura, umidade
MQ135	1	módulo com pinos VCC, GND, DO e AO	gases (índice)
Resistor 4,7 kΩ	3	1/4 W	1 pull-up do DHT22 e 2 no divisor do MQ135
Protoboard	1	830 furos	montagem sem solda
Jumpers	~20	macho-macho e macho-fêmea	ligações
Cabo USB	1	micro-USB **de dados** (não apenas de carga)	gravação, Monitor Serial, alimentação
Fonte	1	5 V / 3 A com saída USB	alimentação sem computador (Etapa 10)
Multímetro	1	com modos de continuidade e tensão contínua	verificações de segurança

Fonte: Elaborado pelos autores.

Foram recebidos dois DHT22, mas decidiu-se usar apenas um, já que o BME280 também mede temperatura e umidade e serve de comparação. Resistores de 1 kΩ também estavam disponíveis, mas não foram necessários.

Figura 14 – Placa adaptadora do PMS7003 usada no protótipo. Foi usado o conector P3, de 4 pinos com passo de 2,54 mm.



Fonte: Registro fotográfico do projeto.

A placa adaptadora do PMS7003 tem três conectores:

- **P1** (fêmea 2×5, preto): onde o sensor é encaixado;
- **P2** (branco, com cabo de 8 vias): para aparelhos que usam o conector original; **não usado**;
- **P3** (4 pinos de 2,54 mm): ligação ao ESP32. Ordem confirmada na serigrafia: **TX, RX, VCC, GND**.

Os sinais SET (ativa/hiberna) e RESET não são ligados, pois o sensor fica ativo por padrão.

3.2 Ferramentas e ambiente

O desenvolvimento foi feito em um computador de laboratório com **Ubuntu 24.04**, usando a **Arduino IDE 1.8.19** (instalada pelo App Center, em formato *snap*) com o pacote de placas **esp32 by Espressif Systems, versão 2.0.17**. A versão mais recente do pacote (3.3.12, na época) não funcionou com essa IDE por causa da versão antiga do Python embutida no *snap* (Capítulo 7). A linguagem é C++ com o núcleo Arduino para ESP32. Para a nuvem, foi usada uma conta gratuita do ThingSpeak e, como rede com internet, o roteador (hotspot) de um celular.

3.3 Metodologia de montagem incremental

Como requisito do projeto, a montagem não poderia ter erros. Adotou-se então uma metodologia incremental, com as seguintes regras, aplicadas em todas as etapas:

1. **Um sensor por etapa.** Cada sensor só é acrescentado depois que o anterior foi aprovado. Se algo falha, a causa está no último elemento adicionado.
2. **Montar com tudo desligado.** Qualquer fio é colocado ou retirado com o cabo USB desconectado.
3. **Verificar antes de energizar.** Com o multímetro em continuidade: 3V3–GND e VIN–GND **não podem apitar** (curto-circuito); cada fio de sinal novo **tem que apitar** entre o pino do sensor e o pino do ESP32.
4. **Medir tensões críticas.** No MQ135, a tensão do ponto médio do divisor foi medida antes de ser ligada ao ESP32.

5. **Um programa novo por etapa**, sempre em um *sketch* novo (File → New), nunca como aba de outro *sketch*.
6. **Critério objetivo de aprovação**, com um "teste de vida" que prova que o sensor reage ao fenômeno (mão, sopro, poeira, álcool).

Tabela 10 – Etapas do desenvolvimento e critérios de aprovação.

Etapa	Conteúdo	Critério de aprovação
1	Inventário e identificação das peças	modelos, versões e tensões confirmados
2	Mapa de ligações (planejamento)	todos os pinos definidos e conferidos
3	Arduino IDE e teste do ESP32 sozinho	gravação OK, dados do chip no Monitor Serial, LED piscando
4	DHT22	leituras plausíveis e reação à mão
5 / 5b	BME280 (+ DHT22)	sensor encontrado no I²C, ID conferido, valores coerentes com o DHT22
6	PMS7003	pacotes válidos, 0 pacotes ruins, reação à poeira
7	MQ135 com divisor	ponto médio < 2,5 V medido; reação ao álcool
8	Estação integrada	4 sensores sem falhas; recuperação automática de falha
9	Versão final: página Wi-Fi + ThingSpeak	página no celular e dados chegando à nuvem
10	Alimentação por fonte USB 5 V	funcionamento sem computador

Fonte: Elaborado pelos autores.

3.4 Preparação do ambiente de software, do zero

3.4.1 Instalação da Arduino IDE

Há duas formas de instalar a IDE no Ubuntu sem complicação. A **forma usada no projeto** foi o App Center do Ubuntu (pesquisar "Arduino IDE" e instalar), que entrega a versão 1.8.19 em *snap*. A alternativa é a Arduino IDE 2, baixada de `arduino.cc/en/software` no formato "Linux ZIP file 64 bits", que não exige privilégios de administrador:

```
cd ~/Downloads
unzip arduino-ide_*_Linux_64bit.zip
cd arduino-ide_*_Linux_64bit
./arduino-ide --no-sandbox        # no Ubuntu 24.04 sem sudo, sem esta opção a janela não abre
```

3.4.2 Pacote de placas ESP32

1. Em **File → Preferences**, no campo *Additional boards manager URLs*, colar: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
2. Em **Tools → Board → Boards Manager**, pesquisar **esp32** (by Espressif Systems), escolher a versão **2.0.17** no menu de versões e clicar em **Install**. O pacote é grande e a IDE 1.8 pode parecer travada por 5 a 10 minutos; não feche a janela até aparecer "INSTALLED".
3. Em **Tools → Board → esp32**, escolher **ESP32 Dev Module**.

> **Versão validada.** Todos os programas deste documento foram compilados, gravados e testados com o pacote esp32 **2.0.17**. Para reproduzir a estação exatamente como foi validada, use essa versão. As versões 3.x do pacote mudaram parte da API do ESP32; elas não foram testadas neste protótipo.

Tabela 11 – Configuração da placa usada na gravação (menu Tools).

Opção	Valor
Board	ESP32 Dev Module
Flash Size	4MB (32Mb)
Partition Scheme	Default 4MB with spiffs (1.2MB APP / 1.5MB SPIFFS)
CPU Frequency	240 MHz (WiFi/BT)
Flash Mode / Frequency	QIO / 80 MHz
Upload Speed	115200
Port	/dev/ttyACM0

Fonte: Configuração registrada na barra de status da IDE durante o projeto.

3.4.3 Acesso à porta USB no Linux

Ao conectar a placa, o comando `lsusb` deve listar o conversor `QinHeng Electronics ... 1a86:55d4` e a porta `/dev/ttyACM0` deve aparecer. Por padrão, essa porta pertence ao grupo `dialout`, e usuários fora dele recebem erro de permissão ao gravar (a IDE 1.8 mostra a mensagem enganosa "port doesn't exist"). Há duas soluções, ambas exigindo `sudo` uma única vez:

```
# Diagnóstico
lsusb                       # deve aparecer: QinHeng Electronics ... 1a86:55d4
ls -l /dev/ttyACM*          # deve existir /dev/ttyACM0
groups                      # o usuário está no grupo dialout?

# Solução 1: incluir o usuário no grupo dialout (vale após sair e entrar na sessão)
sudo usermod -aG dialout $USER

# Solução 2 (usada no projeto): regra udev que libera somente esta placa, permanente
echo 'SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="55d4", MODE="0666"' | sudo
tee /etc/udev/rules.d/99-esp32.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
# desconectar e reconectar a placa; conferir: ls -l /dev/ttyACM0  ->  crw-rw-rw-
```

Placas com o conversor CH340 aparecem como `/dev/ttyUSB0` e, no Ubuntu, podem ter a porta tomada pelo serviço `brltty` (leitor braille), que pode ser removido com `sudo apt remove brltty` quando não for usado. Isso não se aplicou à placa deste projeto.

3.4.4 Bibliotecas

As bibliotecas externas são instaladas em **Sketch → Include Library → Manage Libraries**. Quando a IDE perguntar sobre dependências, deve-se escolher **Install all**.

Tabela 12 – Bibliotecas utilizadas.

Biblioteca	Origem	Usada em	Para quê
DHT sensor library	Adafruit (Library Manager)	Etapas 4 a 9	protocolo do DHT22
Adafruit Unified Sensor	Adafruit (dependência)	Etapas 4 a 9	interface comum de sensores
Adafruit BMP280 Library	Adafruit (Library Manager)	Etapa 5	primeiro teste do sensor de pressão
Adafruit BusIO	Adafruit (dependência)	Etapas 5 a 9	acesso ao I²C
Adafruit BME280 Library	Adafruit (Library Manager)	Etapas 5b a 9	leitura e compensação do BME280
Wire	pacote esp32	Etapas 5 a 9	barramento I²C
WiFi, WebServer, HTTPClient	pacote esp32	Etapa 9	rede própria, página web e envio ao ThingSpeak
(nenhuma)	código próprio	Etapas 6 a 9	leitura do PMS7003 e do MQ135

Fonte: Elaborado pelos autores.

3.4.5 Rotina de gravação e teste

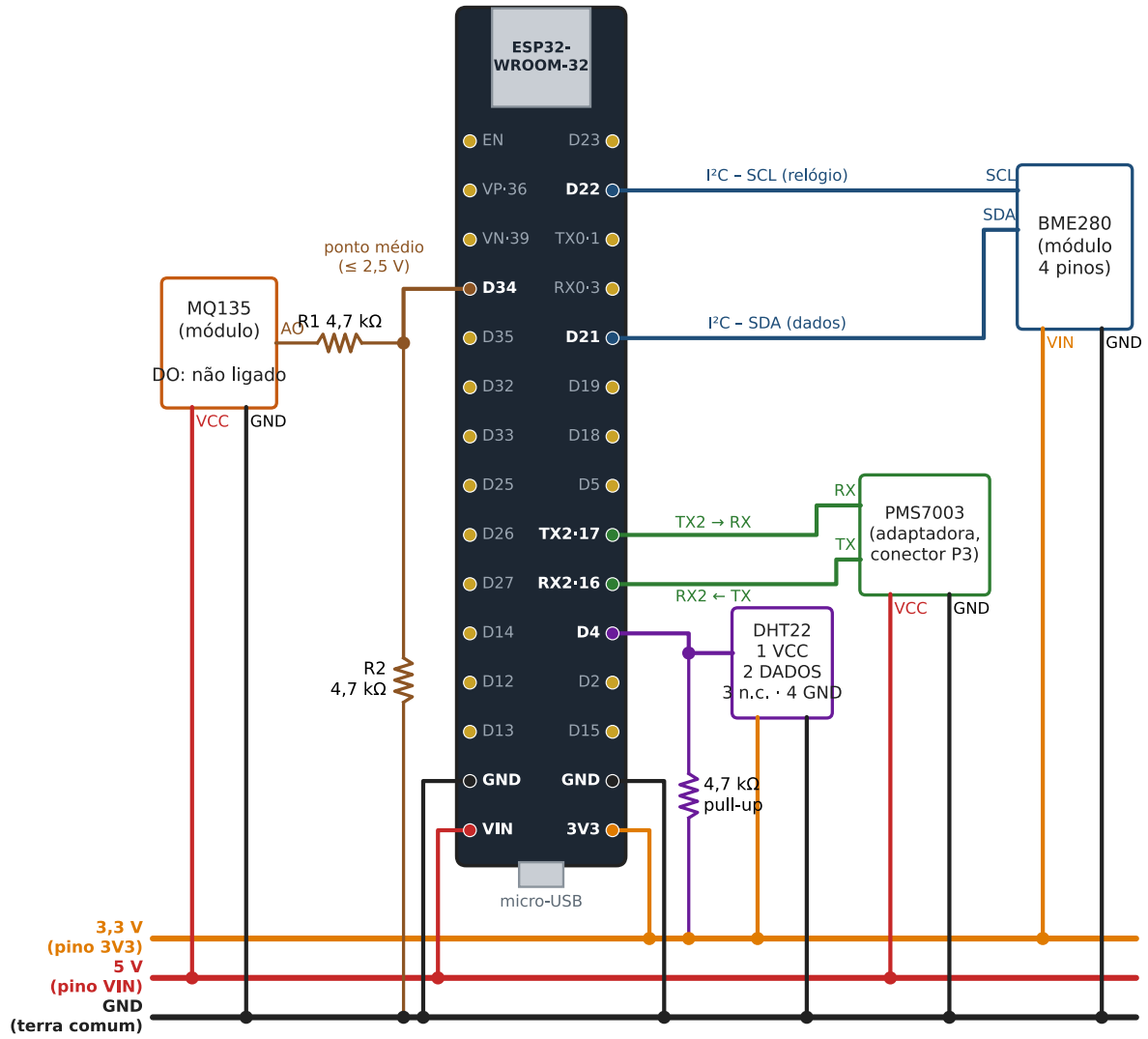
1. **File → New**; apagar o conteúdo, colar o programa da etapa e salvar com o nome indicado.
2. Clicar em ✓ **Verify**. O sucesso aparece como "Sketch uses ... bytes".
3. Conferir **Tools → Port → /dev/ttyACM0** e clicar na **seta (Upload)**. Se travar em `Connecting....`, segurar o botão **BOOT** até aparecer `Writing at 0x...`.
4. Abrir **Tools → Serial Monitor** a **115200 baud** e apertar **EN** para ver as mensagens iniciais.

3.5 Mapa de ligações

O mapa de ligações foi definido antes da montagem (Etapa 2) e não mudou até o fim. A Figura 15 mostra o esquema completo, e a Tabela 13 lista cada ligação.

Figura 15 – Esquema de ligações completo do protótipo. Os trilhos inferiores representam as linhas de 3,3 V, 5 V e GND da protoboard.

Disposição de pinos da DevKit de 30 pinos (vista de cima, antena para cima). Confira sempre a serigrafia da sua placa.



Fonte: Elaborado pelos autores.

Tabela 13 – Tabela de ligações.

Componente	Pino do componente	Pino do ESP32	Observação
PMS7003 (P3)	TX	RX2 / GPIO16	ligação cruzada
	RX	TX2 / GPIO17	ligação cruzada
	VCC	VIN (5 V)	ventoinha e laser precisam de 5 V
	GND	GND	
BME280	VIN	3V3	aceitaria 5 V, mas 3,3 V garante sinais de 3,3 V
	GND	GND	
	SCL	D22 / GPIO22	I²C relógio
	SDA	D21 / GPIO21	I²C dados
DHT22	1 (VCC)	3V3	

Componente	Pino do componente	Pino do ESP32	Observação
	2 (dados)	D4 / GPIO4	+ resistor 4,7 kΩ entre o pino 2 e o 3V3
	3	—	não ligar
	4 (GND)	GND	
MQ135	VCC	VIN (5 V)	aquecedor precisa de 5 V
	GND	GND	
	DO	—	não ligar
	AO	—	**nunca direto no ESP32**: AO → 4,7 kΩ → ponto médio → 4,7 kΩ → GND
Divisor	ponto médio	D34 / GPIO34	no máximo 2,5 V

Fonte: Elaborado pelos autores.

Regras de segurança. Nunca ligar a fonte no VIN e o cabo USB ao mesmo tempo. Nunca ligar a saída AO do MQ135 diretamente em um pino do ESP32. Nunca inverter VCC e GND (em especial no PMS7003, cujos pinos ficam lado a lado no conector P3; usar jumper vermelho no VCC e preto no GND).

4 Desenvolvimento por etapas

Este capítulo descreve as etapas práticas na ordem em que foram executadas. Para cada uma, são apresentados o objetivo, a montagem, o trecho essencial do programa (os códigos completos estão no Apêndice A), o resultado real obtido e a análise. As Etapas 1 e 2 (inventário e mapa) já foram cobertas no Capítulo 3.

4.1 Etapa 3: o ESP32 sozinho

Objetivo: provar que a IDE, o pacote da placa, o cabo, a porta e a gravação funcionam, sem nenhum sensor ligado, para que um problema de computador não seja confundido com um problema de montagem. O programa imprime as informações do chip com as funções da classe `ESP` e pisca o LED azul do GPIO2 a cada segundo.

Listagem 1 – Programa de teste da Etapa 3 (etapa03_teste_esp32.ino).

```
// Etapa 3: teste do ESP32 sozinho (sem nenhum sensor ligado)
// - Pisca o LED azul da placa (GPIO2)
// - Mostra informações do chip no Monitor Serial (115200 baud)

const int LED_PLACA = 2;  // LED azul embutido na DevKit de 30 pinos

void setup() {
  Serial.begin(115200);
  delay(1000);  // tempo para o Monitor Serial abrir

  pinMode(LED_PLACA, OUTPUT);

  Serial.println();
  Serial.println("=== Etapa 3: teste do ESP32 ===");
  Serial.print("Modelo do chip: ");
  Serial.println(ESP.getChipModel());
  Serial.print("Revisao: ");
  Serial.println(ESP.getChipRevision());
  Serial.print("Nucleos: ");
  Serial.println(ESP.getChipCores());
  Serial.print("Frequencia (MHz): ");
  Serial.println(ESP.getCpuFreqMHz());
  Serial.print("Memoria flash (MB): ");
  Serial.println(ESP.getFlashChipSize() / (1024 * 1024));
  Serial.println("Se voce esta lendo isto, a placa esta funcionando!");
}

void loop() {
  digitalWrite(LED_PLACA, HIGH);
  Serial.println("LED aceso");
  delay(1000);

  digitalWrite(LED_PLACA, LOW);
  Serial.println("LED apagado");
  delay(1000);
}
```

Figura 16 – Resultado real da Etapa 3 no Monitor Serial: ESP32-D0WD-V3, revisão 3, 2 núcleos, 240 MHz e 4 MB de flash.



Fonte: Registro do Monitor Serial do projeto.

O resultado confirmou o hardware descrito no inventário. O programa ocupou 273 192 bytes (20 % da área de programa). Esta etapa foi a mais trabalhosa do projeto por causa do ambiente Linux do laboratório; os problemas e as soluções estão no Capítulo 7.

4.2 Etapa 4: DHT22

Montagem: perna 1 no 3V3, perna 2 no GPIO4 com o resistor de 4,7 kΩ até o 3V3, perna 3 livre e perna 4 no GND. A orientação é a da grade de furos virada para quem olha, com as pernas para baixo, numeradas da esquerda para a direita. **Verificação:** 3V3–GND não apita; perna 2–GPIO4 apita; perna 4–GND apita.

O programa usa a biblioteca da Adafruit. As funções `readHumidity()` e `readTemperature()` devolvem `NAN` (não é um número) quando a leitura falha, por exemplo por soma de verificação incorreta; o programa testa isso com `isnan()`. O intervalo de 2,5 s respeita o mínimo de 2 s do sensor.

Listagem 2 – Trecho essencial da Etapa 4.

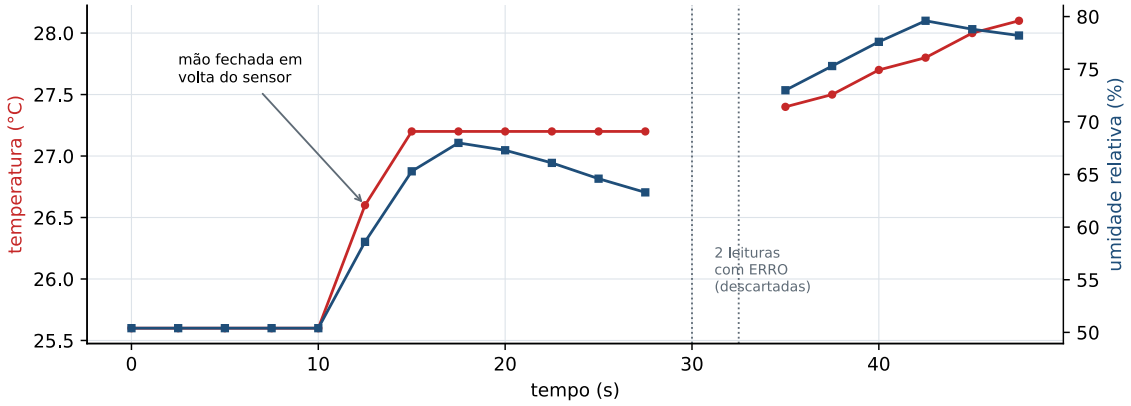
```
DHT dht(PINO_DHT, DHT22);                  // objeto do sensor no GPIO4
...
float umidade = dht.readHumidity();
float temperatura = dht.readTemperature();  // graus Celsius
if (isnan(umidade) || isnan(temperatura)) {
  Serial.println("ERRO: falha ao ler o DHT22 (confira as ligacoes e o resistor)");
}
delay(2500);                                // o DHT22 aceita 1 leitura a cada 2 s
```

Figura 17 – Resultado real da Etapa 4. Com o sensor fechado na mão, temperatura e umidade sobem.



Fonte: Registro do Monitor Serial do projeto.

Figura 18 – Dados reais da Figura 17 em gráfico (uma leitura a cada 2,5 s).



Fonte: Dados do projeto, transcritos do Monitor Serial.

Análise: em repouso, 25,6 °C e 50,4 %, valores típicos de ambiente interno. Ao fechar a mão em volta do sensor, a umidade subiu de 50,4 % para 79,6 % e a temperatura de 25,6 °C para 28,1 °C, provando que os dois elementos sensores respondem. As duas leituras com erro ocorreram enquanto os fios eram manipulados e foram descartadas corretamente; no firmware final, falhas assim são contadas e ignoradas. A etapa foi aprovada.

4.3 Etapa 5: sensor de pressão no barramento I²C

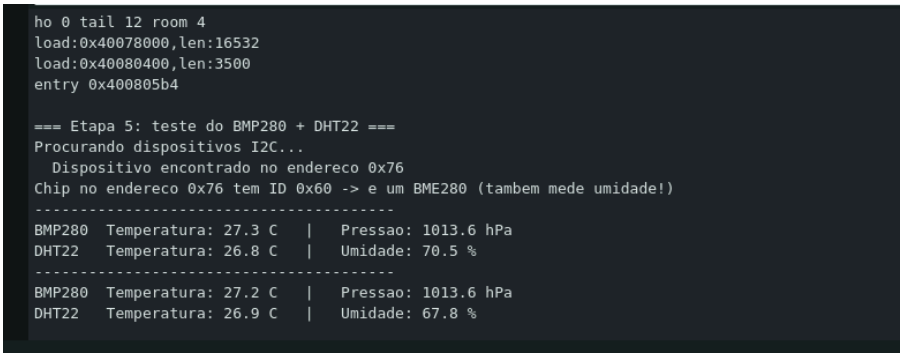
Montagem: VIN no 3V3 (a mesma linha do DHT22), GND no GND, SCL no GPIO22 e SDA no GPIO21. Trocar SDA e SCL não danifica nada, mas impede a comunicação. O DHT22 permaneceu ligado.

Como a identificação visual do chip (BMP280 ou BME280) é incerta, o programa foi escrito para **descobrir o sensor sozinho**: primeiro faz uma varredura de todos os endereços I²C (o "scanner"), depois lê o registrador de identificação 0xD0 em 0x76 e 0x77, como simulado na Figura 6.

Listagem 3 – Trecho da Etapa 5: varredura I²C e identificação do chip.

```
// Le o registrador de identificacao (0xD0): 0x58 = BMP280, 0x60 = BME280
uint8_t lerChipId(uint8_t endereco) {
  Wire.beginTransmission(endereco);
  Wire.write(0xD0);
  if (Wire.endTransmission() != 0) return 0;   // ninguem respondeu (sem ACK)
  Wire.requestFrom((int)endereco, 1);
  if (Wire.available()) return Wire.read();
  return 0;
}
...
for (uint8_t endereco = 1; endereco < 127; endereco++) {   // varredura do barramento
  Wire.beginTransmission(endereco);
  if (Wire.endTransmission() == 0) { /* dispositivo encontrado */ }
}
```

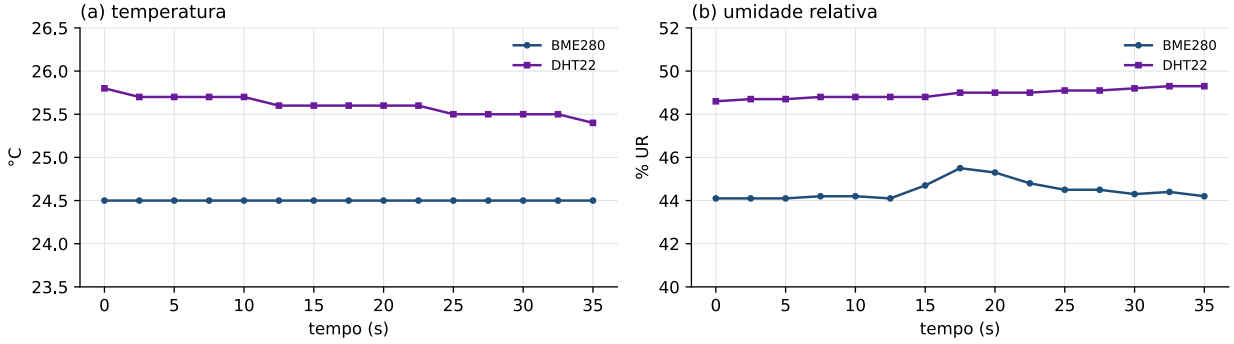
Figura 19 – Resultado real da Etapa 5: dispositivo em 0x76 com ID 0x60, ou seja, um BME280.



Fonte: Registro do Monitor Serial do projeto.

Resultado e decisão: o chip respondeu 0x60. O sensor é um **BME280**, que também mede umidade. Como a biblioteca do BMP280 não lê umidade, foi criada a Etapa 5b, com a biblioteca **Adafruit BME280 Library**, sem mexer em nenhum fio. A função `bme.begin(0x76, &Wire)` tenta o endereço 0x76 e, se falhar, 0x77.

Figura 20 – Dados reais da Etapa 5b após estabilização: BME280 e DHT22 lado a lado (leituras a cada 2,5 s).



Fonte: Dados do projeto, transcritos do Monitor Serial.

Análise: logo após o teste com a mão, a diferença de temperatura chegou a 2,3 °C, porque o DHT22 ainda estava aquecido. Depois de 10 minutos sem toque, as leituras ficaram em 24,5 °C (BME280) e 25,4 °C (DHT22), diferença de 0,9 °C, e em 44 % e 49 % de umidade. Considerando

as exatidões de fábrica (±1 °C e ±3 % para o BME280; ±0,5 °C e ±2 a 5 % para o DHT22), as diferenças estão dentro do esperado. A pressão ficou estável em 1013,6 hPa, valor coerente com um local próximo ao nível do mar (Figura 8). O BME280 foi adotado como referência de temperatura, umidade e pressão por ser o mais exato; o DHT22 permanece como segunda opinião.

4.4 Etapa 6: PMS7003

Montagem: pelo conector P3 da adaptadora: TX no GPIO16 (RX2), RX no GPIO17 (TX2), VCC no **VIN** e GND no GND. É o primeiro sensor ligado em 5 V. **Verificação:** VIN–GND e 3V3–GND não apitam; VCC–VIN e GND–GND apitam. Ao energizar, a ventoinha do sensor começa a girar, com um zumbido baixo.

Programa: nenhuma biblioteca foi usada; o pacote é lido e validado pelo próprio código, o que torna o funcionamento transparente. A UART2 é iniciada com `Serial2.begin(9600, SERIAL_8N1, 16, 17)`. A função `lerPms()` é uma pequena máquina de estados: descarta bytes até encontrar 0x42, exige 0x4D em seguida, junta os 32 bytes, confere o comprimento (28) e a soma de verificação e só então extrai os valores.

Listagem 4 – Trecho da Etapa 6: máquina de estados que monta e valida o pacote do PMS7003.

```
if (idx == 0) {                       // esperando o primeiro byte do cabecalho
  if (b == 0x42) buf[idx++] = b;
  continue;
}
if (idx == 1 && b != 0x4D) {          // segundo byte errado: recomeca
  idx = (b == 0x42) ? 1 : 0;
  continue;
}
buf[idx++] = b;
if (idx < 32) continue;               // ainda nao chegou o pacote inteiro
idx = 0;
uint16_t comprimento = (buf[2] << 8) | buf[3];
uint16_t soma = 0;
for (int i = 0; i < 30; i++) soma += buf[i];
uint16_t verificacao = (buf[30] << 8) | buf[31];
if (comprimento != 28 || soma != verificacao) { pacotesRuins++; continue; }
d.pm2_5 = (buf[12] << 8) | buf[13];   // PM2.5 "ambiente atmosferico"
```

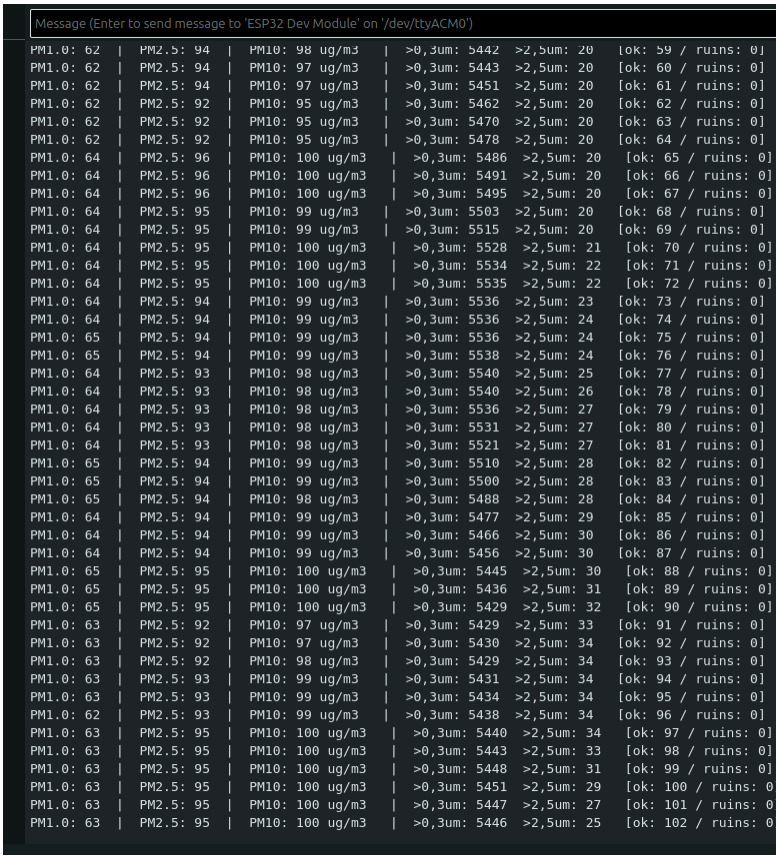
Tabela 14 – Simulação: pacote de 32 bytes montado com os valores reais da primeira leitura da Etapa 6 (PM1,0 = 62, PM2,5 = 94, PM10 = 98 µg/m³; 5442 partículas >0,3 µm; 20 >2,5 µm), em hexadecimal.

Bytes	+0	+1	+2	+3	+4	+5	+6	+7
0–7	42 início	4D início	00 compr.	1C compr.	00 PM1 CF1	3F	00 PM2,5 CF1	60
8–15	00 PM10 CF1	65	00 PM1 atm	3E	00 PM2,5 atm	5E	00 PM10 atm	62
16–23	15 >0,3 µm	42	06 >0,5 µm	40	01 >1,0 µm	2C	00 >2,5 µm	14
24–31	00 >5 µm	04	00 >10 µm	01	97 reserv.	00	04 verif.	27

Fonte: Elaborado pelos autores com o script de simulação; os campos CF=1 e as contagens >0,5, >1, >5 e >10 µm são fictícios.

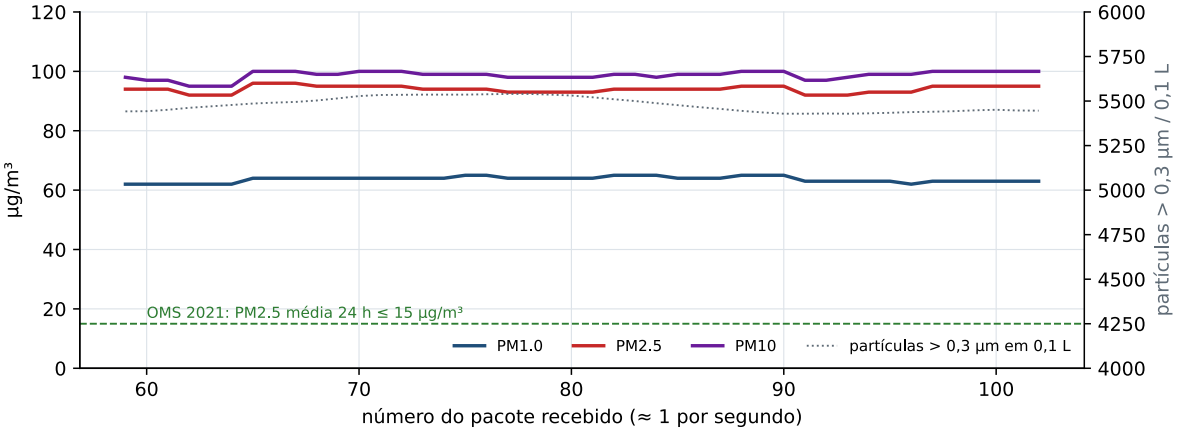
Aplicando ao pacote acima o mesmo algoritmo do firmware, a soma dos bytes 0 a 29 é 1063 (0x0427), igual ao valor dos bytes 30–31, e o comprimento é 28: o pacote é aceito e decodificado como PM1,0 = 62, PM2,5 = 94 e PM10 = 98 µg/m³. Se um único bit do PM2,5 for corrompido na transmissão (simulação), o valor lido viraria 90, mas a soma passaria a 1059, diferente da verificação, e o pacote seria contado como "ruim" e descartado. É assim que o firmware evita exibir valores corrompidos.

Figura 21 – Resultado real da Etapa 6: mais de 100 pacotes válidos e nenhum pacote ruim.



Fonte: Registro do Monitor Serial do projeto.

Figura 22 – Dados reais da Figura 21 em gráfico, comparados com a diretriz da OMS para média de 24 h.



Fonte: Dados do projeto, transcritos do Monitor Serial.

Análise: foram recebidos 185 pacotes sem nenhum erro, o que valida a ligação e o código. O PM2,5 ficou entre 92 e 96 µg/m³ (média de 94,0 µg/m³ e desvio padrão de 1,1 µg/m³ nos 44 pacotes transcritos), PM1,0 em torno de 64 µg/m³ e PM10 em torno de 99 µg/m³. As contagens são coerentes com esses valores (cerca de 5 400 partículas acima de 0,3 µm em 0,1 L), e PM1,0 < PM2,5 < PM10, como exige a definição. O valor é alto para um ambiente interno (cerca de seis vezes a diretriz de 24 h da OMS), mas plausível para o laboratório onde o teste foi feito, um ambiente grande e com muitos objetos acumulados. O teste com um pano empoeirado foi realizado. A comparação com uma estação oficial próxima não foi possível no momento do teste e fica como validação futura.

4.5 Etapa 7: MQ135 com divisor de tensão

Esta foi a etapa mais delicada, porque envolve um sinal que pode chegar a 5 V. Ela foi feita em três tempos:

1. **Montar sem ligar ao ESP32:** VCC no VIN, GND no GND, AO → R1 (4,7 kΩ) → ponto médio → R2 (4,7 kΩ) → GND. O ponto médio ainda **não** é ligado ao GPIO34. Em continuidade, AO–GPIO34 não pode apitar.
2. **Medir com energia:** com o multímetro em tensão contínua, o ponto médio precisa estar abaixo de 2,5 V e o valor em AO deve ser aproximadamente o dobro.
3. **Só então ligar** o ponto médio ao GPIO34, com o USB desconectado, e gravar o programa.

O programa configura o ADC com 12 bits e atenuação de 11 dB, faz a média de 20 leituras de `analogReadMilliVolts()` e multiplica por 2 (o fator do divisor) para obter a tensão em AO. Um alerta é impresso se o pino passar de 2,8 V.

Listagem 5 – Trecho da Etapa 7: leitura analógica com média e correção do divisor.

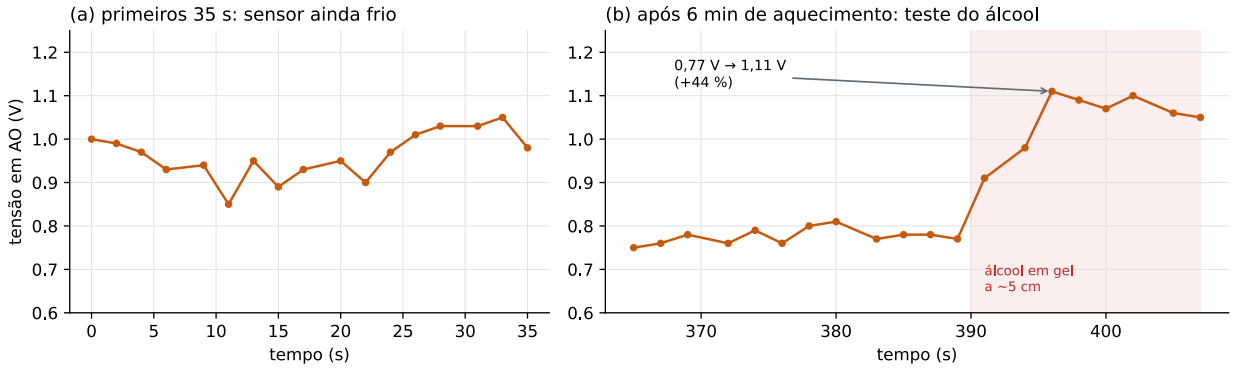
```
analogReadResolution(12);                       // 0 a 4095
analogSetPinAttenuation(PINO_MQ135, ADC_11db);  // faixa de ~0 a 3,1 V
...
for (int i = 0; i < AMOSTRAS; i++) {            // AMOSTRAS = 20
  somaMv += analogReadMilliVolts(PINO_MQ135);   // ja calibrado de fabrica
  delay(10);
}
float mvPino = somaMv / (float)AMOSTRAS;
float vSaidaAO = mvPino * FATOR_DIVISOR / 1000.0;   // FATOR_DIVISOR = 2,0
```

Figura 23 – Resultado real da Etapa 7 durante o teste do álcool (após 6 minutos de aquecimento).

```
Tempo: 365 s  |  Leitura bruta: 289  |  GPIO34: 375 mV  |  Saida AO do MQ135: 0.75 V
Tempo: 367 s  |  Leitura bruta: 298  |  GPIO34: 383 mV  |  Saida AO do MQ135: 0.76 V
Tempo: 369 s  |  Leitura bruta: 307  |  GPIO34: 390 mV  |  Saida AO do MQ135: 0.78 V
Tempo: 372 s  |  Leitura bruta: 297  |  GPIO34: 381 mV  |  Saida AO do MQ135: 0.76 V
Tempo: 374 s  |  Leitura bruta: 315  |  GPIO34: 396 mV  |  Saida AO do MQ135: 0.79 V
Tempo: 376 s  |  Leitura bruta: 296  |  GPIO34: 381 mV  |  Saida AO do MQ135: 0.76 V
Tempo: 378 s  |  Leitura bruta: 320  |  GPIO34: 400 mV  |  Saida AO do MQ135: 0.80 V
Tempo: 380 s  |  Leitura bruta: 326  |  GPIO34: 405 mV  |  Saida AO do MQ135: 0.81 V
Tempo: 383 s  |  Leitura bruta: 298  |  GPIO34: 383 mV  |  Saida AO do MQ135: 0.77 V
Tempo: 385 s  |  Leitura bruta: 309  |  GPIO34: 391 mV  |  Saida AO do MQ135: 0.78 V
Tempo: 387 s  |  Leitura bruta: 306  |  GPIO34: 389 mV  |  Saida AO do MQ135: 0.78 V
Tempo: 389 s  |  Leitura bruta: 303  |  GPIO34: 387 mV  |  Saida AO do MQ135: 0.77 V
Tempo: 391 s  |  Leitura bruta: 390  |  GPIO34: 457 mV  |  Saida AO do MQ135: 0.91 V
Tempo: 394 s  |  Leitura bruta: 430  |  GPIO34: 489 mV  |  Saida AO do MQ135: 0.98 V
Tempo: 396 s  |  Leitura bruta: 514  |  GPIO34: 557 mV  |  Saida AO do MQ135: 1.11 V
Tempo: 398 s  |  Leitura bruta: 503  |  GPIO34: 547 mV  |  Saida AO do MQ135: 1.09 V
Tempo: 400 s  |  Leitura bruta: 486  |  GPIO34: 533 mV  |  Saida AO do MQ135: 1.07 V
Tempo: 402 s  |  Leitura bruta: 505  |  GPIO34: 550 mV  |  Saida AO do MQ135: 1.10 V
Tempo: 405 s  |  Leitura bruta: 483  |  GPIO34: 532 mV  |  Saida AO do MQ135: 1.06 V
Tempo: 407 s  |  Leitura bruta: 477  |  GPIO34: 526 mV  |  Saida AO do MQ135: 1.05 V
```

Fonte: Registro do Monitor Serial do projeto.

Figura 24 – Dados reais do MQ135: (a) início, com o sensor ainda frio; (b) teste com álcool em gel aproximado a cerca de 5 cm.



Fonte: Dados do projeto, transcritos do Monitor Serial.

Análise: o GPIO34 recebeu cerca de 0,5 V, com AO em torno de 1,0 V: o divisor dividiu por dois, como projetado, com ampla margem até o limite de 3,3 V. Nos primeiros minutos, a leitura oscilou entre 0,85 e 1,05 V, o que é normal durante o aquecimento. Depois de seis minutos, a saída estava em cerca de 0,77 V; com o álcool, subiu para 1,11 V (+44 %) em cerca de 5 s e começou a voltar devagar ao retirar o estímulo. Pela Equação 5, e supondo R_L = 1 kΩ, a resistência do sensor caiu de cerca de 5,5 kΩ para 3,5 kΩ (Figura 13b). O sensor reage como esperado e a etapa foi aprovada. Os valores absolutos só terão significado em concentração depois da queima inicial e da calibração.

4.6 Etapa 8: estação integrada

Nenhum fio mudou. O desafio desta etapa é de software: ler quatro sensores com ritmos diferentes sem que um atrapalhe o outro. Um programa ingênuo, com `delay(2500)` para o DHT22, perderia bytes do PMS7003, que chegam continuamente pela UART. A solução foi um **escalonamento cooperativo** baseado em `millis()` (o tempo desde que a placa ligou): o `loop()` nunca espera, só verifica se chegou a hora de cada tarefa (Figura 29).

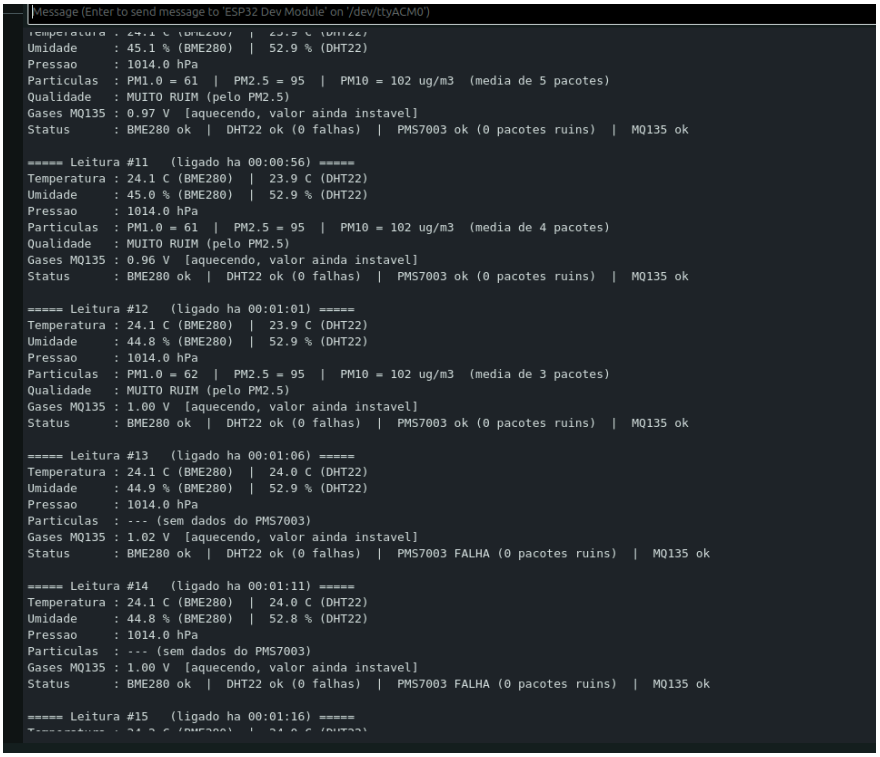
Listagem 6 – Trecho da Etapa 8: tarefas não bloqueantes.

```
void processarDht() {
  if (millis() - dhtUltimaTentativa < INTERVALO_DHT) return;   // ainda nao e hora
  dhtUltimaTentativa = millis();
  float u = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(u) || isnan(t)) { dhtFalhas++; return; }          // descarta a falha
  dhtUmid = u; dhtTemp = t;
  dhtUltimaLeitura = millis();                                // guarda a ultima leitura valida
}

void loop() {
  processarPms();      // le todos os bytes que chegaram
  processarDht();      // no maximo a cada 2,5 s
  processarMq();       // uma amostra a cada 100 ms
  if (millis() - ultimoRelatorio >= INTERVALO_RELATORIO) {    // a cada 5 s
    ultimoRelatorio = millis();
    imprimirRelatorio();
  }
}
```

Cada sensor tem um **prazo de validade**: uma leitura do DHT22 vale por 10 s, e o PMS7003 é considerado em falha se nenhum pacote válido chegar em 5 s. Os valores do PMS7003 e do MQ135 são acumulados e, a cada 5 s, é calculada a média do período. A expressão `millis() - inicio >= intervalo` continua correta mesmo quando o contador de 32 bits reinicia (a cada ~49,7 dias), por causa da aritmética modular dos inteiros sem sinal.

Figura 25 – Resultado real da Etapa 8, incluindo o teste de robustez: nas leituras #13 e #14 o fio TX do PMS7003 foi desconectado, o sensor aparece como FALHA e os demais continuam funcionando.



Fonte: Registro do Monitor Serial do projeto.

Análise: os quatro sensores funcionaram juntos sem falhas do DHT22 e sem pacotes ruins do PMS7003. No teste de robustez, ao desconectar o fio de dados do PMS7003 com o programa rodando, a linha de partículas passou a "sem dados" e o status a FALHA; ao reconectar, tudo voltou a "ok" sozinho, sem reiniciar o ESP32. As temperaturas ficaram praticamente iguais (24,1 e 24,0 °C); a umidade do DHT22 ficou cerca de 8 pontos acima da do BME280, diferença comum para o DHT22, o que reforçou a escolha do BME280 como referência.

4.7 Etapa 9: versão final com página Wi-Fi e ThingSpeak

Entre as opções de destino dos dados (página local, nuvem, ambas ou só USB), foi escolhida a combinação das duas primeiras. O firmware final mantém tudo o que a Etapa 8 fazia e acrescenta:

- **Rede Wi-Fi própria** "EstacaoAr" (senha "qualidade123"), sempre ativa, em que a página `http://192.168.4.1` mostra os valores atuais, a classificação do ar, gráficos das últimas 2 horas e um link para baixar o histórico em CSV. Funciona sem internet.
- **Envio ao ThingSpeak** a cada 20 s, quando uma rede com internet está configurada.

Nenhuma biblioteca externa nova foi necessária: `WiFi.h`, `WebServer.h` e `HTTPClient.h` fazem parte do pacote esp32. Os únicos campos que o usuário precisa editar ficam reunidos no início do programa:

Listagem 7 – Bloco de configuração do programa final.

```
// Rede propria da estacao (sempre ligada)
const char* AP_NOME  = "EstacaoAr";
const char* AP_SENHA = "qualidade123";      // minimo 8 caracteres
// Rede com internet para enviar ao ThingSpeak (deixe "" para nao usar)
const char* WIFI_NOME  = "";
const char* WIFI_SENHA = "";
// ThingSpeak: "Write API Key" do seu canal (deixe "" para nao enviar)
const char* THINGSPEAK_CHAVE = "";
```

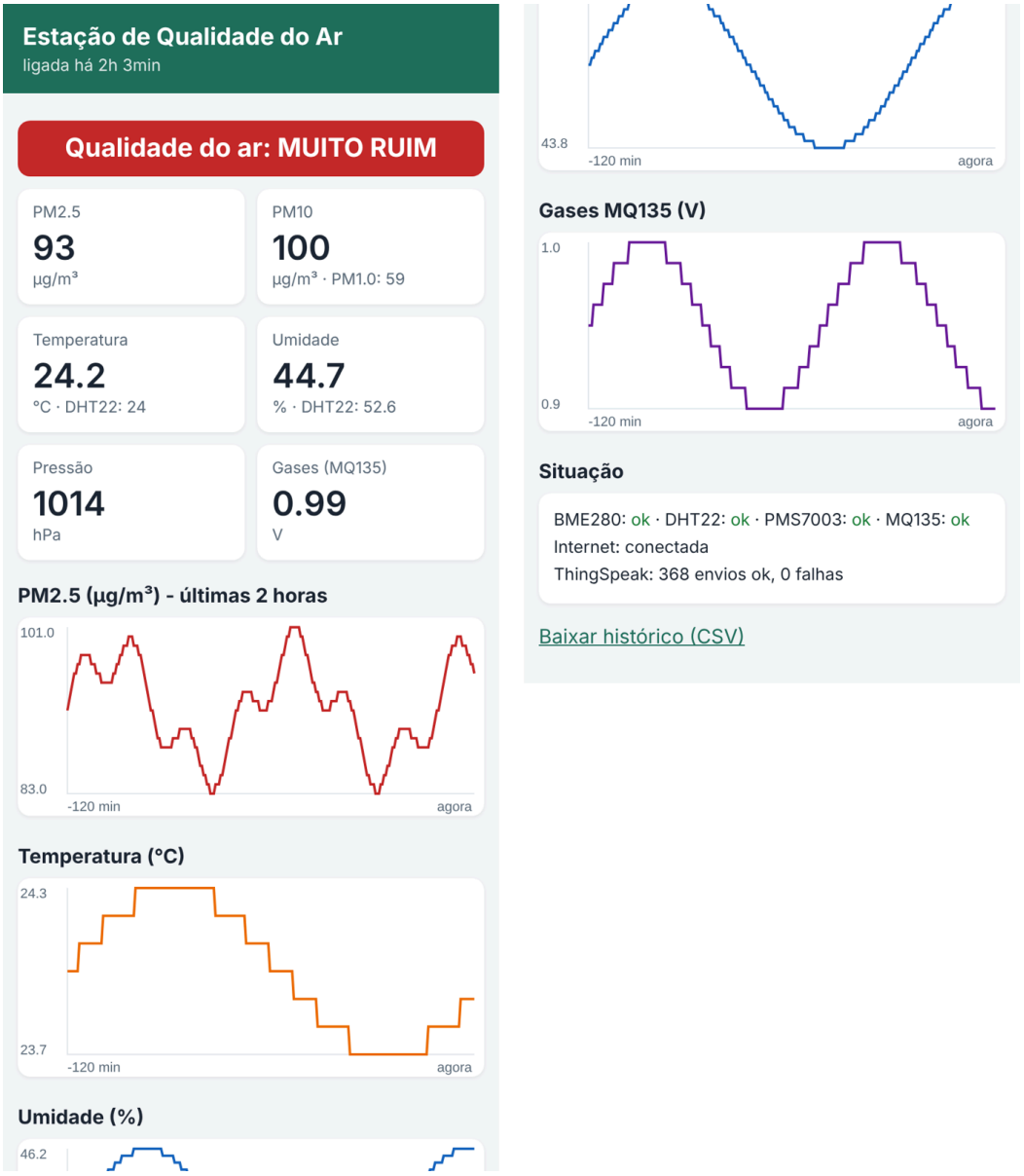
Tabela 15 – Rotas do servidor web do ESP32.

Rota	Resposta	Conteúdo
`/`	text/html	página completa (HTML, CSS e JavaScript gravados na flash com PROGMEM)
`/dados`	application/json	leituras atuais, classificação, status de cada sensor, internet e ThingSpeak
`/historico`	application/json	até 240 pontos (1 a cada 30 s = 2 h) de PM2,5, temperatura, umidade e MQ135
`/csv`	text/csv	o mesmo histórico em arquivo para download (estacao.csv)

Fonte: Elaborado pelos autores.

A página não é recarregada: um código JavaScript consulta `/dados` a cada 5 s e `/historico` a cada 30 s e desenha os gráficos em elementos `<canvas>`, sem bibliotecas externas, o que permite que funcione sem acesso à internet. A Figura 26 mostra a página real do firmware, renderizada em um navegador com dados simulados, próximos aos medidos.

Figura 26 – Página web do firmware final (tela de celular, dividida em duas colunas). Simulação: o HTML é exatamente o gravado no ESP32, alimentado com dados fictícios próximos aos medidos.



Fonte: Elaborado pelos autores, renderizando o HTML do arquivo estacao_final.ino.

4.7.1 Configuração do ThingSpeak

1. Criar uma conta gratuita em `thingspeak.mathworks.com`.
2. Em **Channels → My Channels → New Channel**, criar o canal "Estacao Qualidade do Ar" com os 8 campos da Tabela 16, nessa ordem, e salvar.
3. Na aba **API Keys**, copiar a **Write API Key** e colá-la em `THINGSPEAK_CHAVE`; preencher `WIFI_NOME` e `WIFI_SENHA` e gravar de novo.

Tabela 16 – Campos do canal ThingSpeak.

Campo	Grandeza	Origem
field1	Temperatura (°C)	BME280
field2	Umidade (%)	BME280
field3	Pressão (hPa)	BME280
field4	PM1,0 (µg/m³)	PMS7003
field5	PM2,5 (µg/m³)	PMS7003
field6	PM10 (µg/m³)	PMS7003
field7	MQ135 (V)	MQ135
field8	Umidade DHT22 (%)	DHT22

Fonte: Elaborado pelos autores.

O envio é uma requisição HTTP GET para `http://api.thingspeak.com/update?api_key=CHAVE&field1=...`. Campos de sensores em falha são omitidos, para não gravar valores inválidos na nuvem. O ThingSpeak responde com o número da entrada criada (maior que zero) quando aceita os dados; o firmware conta envios bem-sucedidos e falhas e mostra a última falha na página. O plano gratuito é destinado a uso não comercial e permite 3 milhões de mensagens por ano, 4 canais e um envio a cada 15 s no mínimo; com um envio a cada 20 s, a estação gera cerca de 1,6 milhão de mensagens por ano.

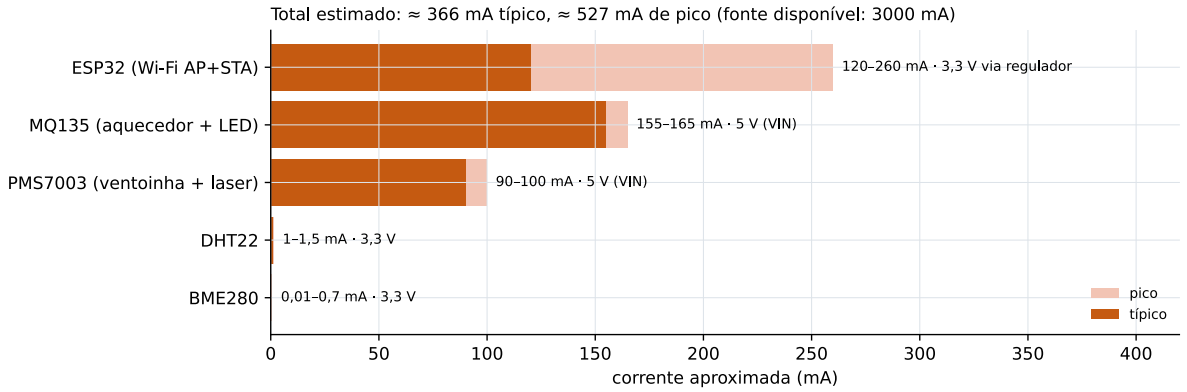
> **Redes compatíveis.** O ESP32 conecta apenas em redes de 2,4 GHz com senha simples (WPA2 pessoal). Redes que pedem usuário e senha pessoais (como a eduroam) ou página de login no navegador não funcionam diretamente. No projeto, a solução foi usar o roteador (hotspot) de um celular, configurado em 2,4 GHz.

Resultado: a página abriu no celular conectado à rede EstacaoAr, mostrando os dados, e o ThingSpeak passou a receber os pontos usando o hotspot do celular como rede com internet. A etapa foi aprovada.

4.8 Etapa 10: alimentação sem computador

Como a fonte disponível (5 V / 3 A) tem saída USB, bastou trocar o computador pela fonte, usando o mesmo cabo micro-USB, sem alterar nenhum fio. Para fontes com plugue P4 ou fios soltos, a ligação seria no VIN (positivo) e no GND (negativo), após conferir a polaridade com o multímetro (+4,8 a +5,3 V) e com o cabo USB desconectado. O programa fica gravado na flash e começa a funcionar sozinho ao ligar.

Figura 27 – Orçamento de corrente estimado do protótipo, a partir dos datasheets e de valores típicos de operação do ESP32 com Wi-Fi.



Fonte: Elaborado pelos autores com base em Espressif (2023a), Plantower (2016), Hanwei (s.d.), Aosong (s.d.) e Bosch Sensortec (2022).

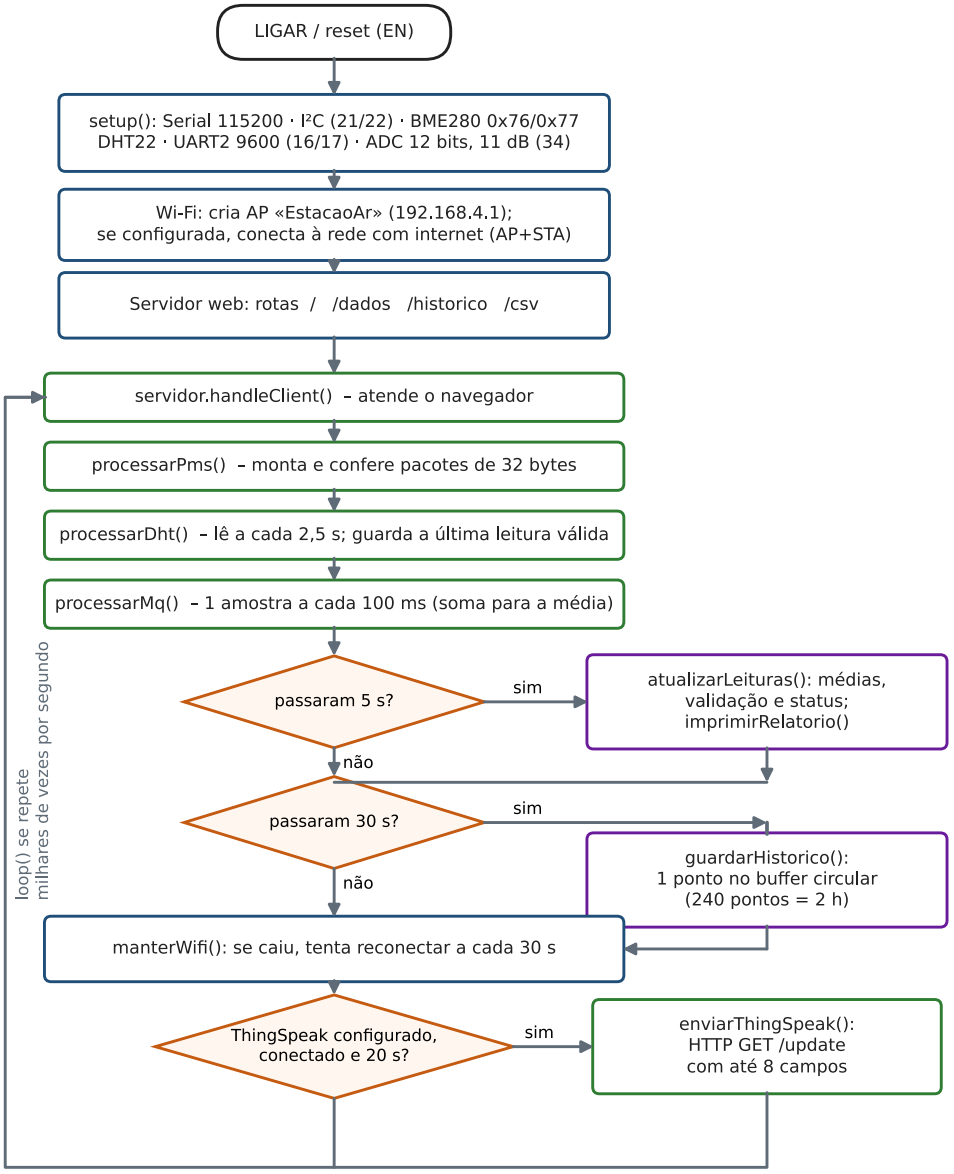
O consumo total, de aproximadamente 0,37 A em operação típica e 0,53 A nos picos de transmissão Wi-Fi, é muito inferior aos 3 A da fonte. Já uma porta USB 2.0 de computador fornece nominalmente até 0,5 A; por isso, em portas fracas, os picos podem derrubar a tensão e reiniciar o ESP32, como previsto nas seções de problemas comuns dos guias. Na DevKit, o pino VIN fica ligado ao 5 V do USB por meio de um diodo, de modo que costuma ficar entre 4,6 e 4,9 V nessa condição.

Para a qualidade das medições, mesmo no protótipo, três cuidados de posicionamento foram adotados: manter o MQ135, que esquenta, a pelo menos 10 cm do BME280 e do DHT22 (e acima deles, pois o ar quente sobe); afastar esses dois sensores também do ESP32, que aquece com o Wi-Fi; e deixar livres a entrada e a saída de ar do PMS7003. O protótipo foi deixado ligado continuamente para a queima inicial do MQ135, que antecede a calibração.

5 Arquitetura do firmware final

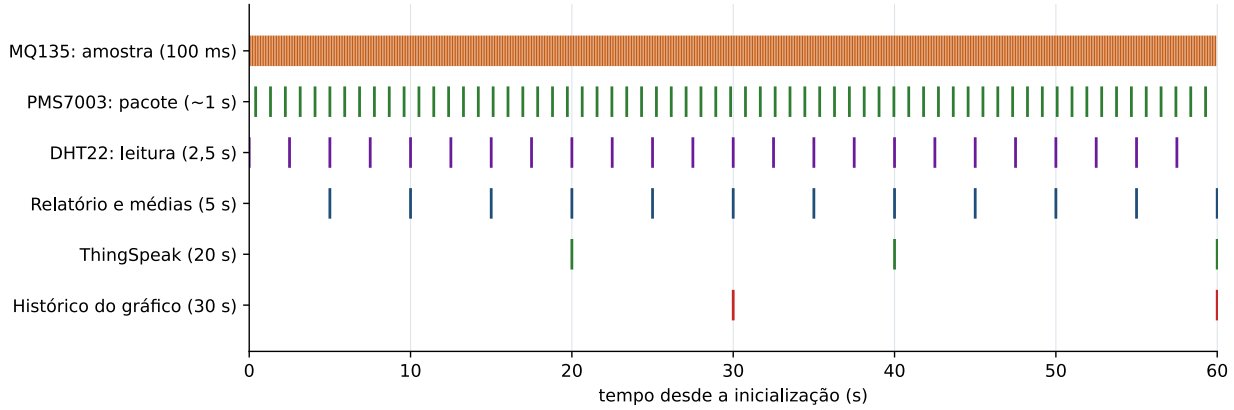
O programa final (`estacao_final.ino`, Listagem completa no Apêndice A) tem cerca de 560 linhas e é organizado em blocos funcionais, descritos a seguir. A Figura 28 mostra o fluxo de execução, e a Figura 29 mostra como as tarefas se distribuem no tempo.

Figura 28 – Fluxograma do firmware final.



Fonte: Elaborado pelos autores.

Figura 29 – Escalonamento cooperativo: instantes em que cada tarefa é executada durante o primeiro minuto de funcionamento.



Fonte: Elaborado pelos autores.

5.1 Organização em blocos

Tabela 17 – Blocos do programa final e suas funções.

Bloco	Funções principais	Responsabilidade
Configuração	constantes no topo	nomes e senhas de rede, chave do ThingSpeak, pinos, intervalos
Leitura dos sensores	`processarPms()`, `processarDht()`, `processarMq()`	coletar dados sem bloquear, validar e acumular
Consolidação	`atualizarLeituras()`	a cada 5 s: médias, faixas válidas, prazos de validade, status
Histórico	`guardarHistorico()`	a cada 30 s: ponto no buffer circular de 240 posições
Monitor Serial	`imprimirRelatorio()`	resumo legível a cada 5 s
Rede	`manterWifi()`, `enviarThingSpeak()`	reconexão a cada 30 s e envio a cada 20 s
Servidor web	`responderPagina()`, `responderDados()`, `responderHistorico()`, `responderCsv()`	página, JSON e CSV

Fonte: Elaborado pelos autores.

5.2 Estruturas de dados

As leituras consolidadas ficam em uma estrutura única, `Leituras atual`, consultada pelo Monitor Serial, pela página e pelo envio à nuvem. Assim, os três mostram sempre os mesmos números. Valores ausentes são representados por `NAN` (ponto flutuante) ou −1 (inteiros) e convertidos em `null` no JSON.

Listagem 8 – Estrutura com as leituras consolidadas.

```
struct Leituras {
  float tBme = NAN, uBme = NAN, pressao = NAN;   // BME280
  float tDht = NAN, uDht = NAN;                  // DHT22
  float mqVolts = NAN;                           // MQ135 (tensao em A0)
  int pm1 = -1, pm25 = -1, pm10 = -1;            // PMS7003
```

```
  bool bmeOk = false, dhtOk = false, pmsOk = false, mqOk = false;
  bool mqAquecendo = true;
};
```

O histórico usa um **buffer circular** de 240 posições: os índices `histInicio` e `histQtd` indicam o ponto mais antigo e a quantidade armazenada. Quando o buffer enche, o ponto mais antigo é sobrescrito, de modo que a memória usada é fixa (240 × 20 bytes ≈ 4,8 KB) e o histórico cobre sempre as últimas 2 horas. Os tempos são registrados em segundos desde a inicialização, porque o protótipo não tem relógio de tempo real (Capítulo 8).

5.3 Regras de validação

Tabela 18 – Critérios usados pelo firmware para considerar cada leitura válida.

Sensor	Leitura considerada válida quando	Se inválida
BME280	o sensor iniciou e 300 ≤ pressão ≤ 1100 hPa e a temperatura não é NAN	status FALHA; campos omitidos
DHT22	houve leitura sem erro nos últimos 10 s	mantém a última válida por até 10 s; depois, FALHA
PMS7003	comprimento = 28, soma de verificação correta e pacote recebido nos últimos 5 s	pacote descartado e contado; após 5 s, FALHA
MQ135	média do pino entre 20 e 2800 mV	FALHA (indica divisor desligado ou errado)
MQ135 (aquecimento)	mais de 3 minutos desde a inicialização	valor exibido com aviso "aquecendo"

Fonte: Elaborado pelos autores.

5.4 Comunicação com a nuvem e robustez da rede

O modo Wi-Fi é definido em `setup()`: apenas AP quando não há rede configurada, ou AP+STA quando há. A reconexão automática do driver é ativada e, como camada adicional, `manterWifi()` refaz a conexão a cada 30 s se ela cair. O envio ao ThingSpeak usa tempo limite de 4 s, para que uma rede lenta não congele a estação. Se a internet cair, a página local continua funcionando e os envios recomeçam sozinhos quando a rede volta.

6 Resultados e discussão

6.1 Síntese dos resultados

4 / 4	0	0,9 °C	+44 %
sensores operando juntos, sem falhas na Etapa 8	pacotes ruins do PMS7003 em 185 recebidos (Etapa 6)	diferença BME280 × DHT22 após estabilização	resposta do MQ135 ao vapor de álcool

Tabela 19 – Valores medidos no laboratório durante as etapas.

Grandeza	Sensor	Valor medido	Observação
Temperatura	BME280	24,1 a 24,5 °C	referência
Temperatura	DHT22	24,0 a 25,6 °C	concorda com o BME280 dentro de ~1 °C
Umidade relativa	BME280	44 a 45 %	referência
Umidade relativa	DHT22	49 a 53 %	5 a 8 pontos acima do BME280
Pressão	BME280	1013,6 a 1014,0 hPa	estável; compatível com local próximo ao nível do mar
PM1,0	PMS7003	55 a 65 µg/m³	
PM2,5	PMS7003	84 a 96 µg/m³ (média 94)	classificação MUITO RUIM pelo índice
PM10	PMS7003	90 a 102 µg/m³	
Tensão AO (aquecido)	MQ135	≈ 0,77 V em repouso; 1,11 V com álcool	índice relativo, sem calibração

Fonte: Dados do projeto (Etapas 4 a 8).

6.2 Discussão

6.2.1 Concordância entre os sensores de temperatura e umidade

Ter dois sensores independentes para a mesma grandeza permitiu uma verificação cruzada simples e valiosa. As diferenças finais (0,9 °C e 5 a 8 pontos de umidade) são compatíveis com as tolerâncias somadas dos dois sensores. O desvio sistemático de umidade do DHT22 é um comportamento conhecido desse modelo e justifica o uso do BME280 como referência. Diferenças maiores logo após manipulações mostram outra lição prática: o tempo de resposta térmico dos encapsulamentos é de minutos, e o calor das mãos ou do ESP32 influencia as leituras. Por isso, qualquer comparação deve ser feita após estabilização.

6.2.2 Material particulado no laboratório

O PM2,5 de cerca de 94 µg/m³ é elevado para um ambiente interno. Três elementos indicam que se trata de uma medida real, e não de defeito: a comunicação estava perfeita (0 pacotes ruins), as três frações respeitaram a ordem física PM1,0 < PM2,5 < PM10 e as contagens de partículas

foram coerentes com as massas. Além disso, a umidade estava em cerca de 45 %, abaixo da faixa em que o crescimento higroscópico das partículas distorce a medida óptica. Ainda assim, sensores de baixo custo têm erro de consistência de ±10 µg/m³ ou ±10 % e podem apresentar desvios sistemáticos dependentes do tipo de partícula; uma comparação lado a lado com uma estação de referência é necessária para atribuir exatidão aos valores absolutos.

6.2.3 Sensor de gases

O MQ135 mostrou o comportamento qualitativo esperado: queda de leitura durante o aquecimento, estabilização e resposta rápida e reversível ao álcool. Os números, porém, são tensões e dependem do resistor de carga do módulo, da temperatura, da umidade e do envelhecimento do sensor. Sem a queima inicial e a calibração em ar limpo, eles devem ser lidos apenas como tendência (subiu ou desceu).

6.2.4 Robustez do sistema

O teste de desconexão do PMS7003 mostrou que o firmware isola as falhas: um sensor ausente não trava os demais, não exibe valores antigos como se fossem atuais e volta a operar sozinho. Esse comportamento decorre de três escolhas de projeto: leitura não bloqueante, prazo de validade das leituras e validação por soma de verificação.

7 Dificuldades encontradas e soluções

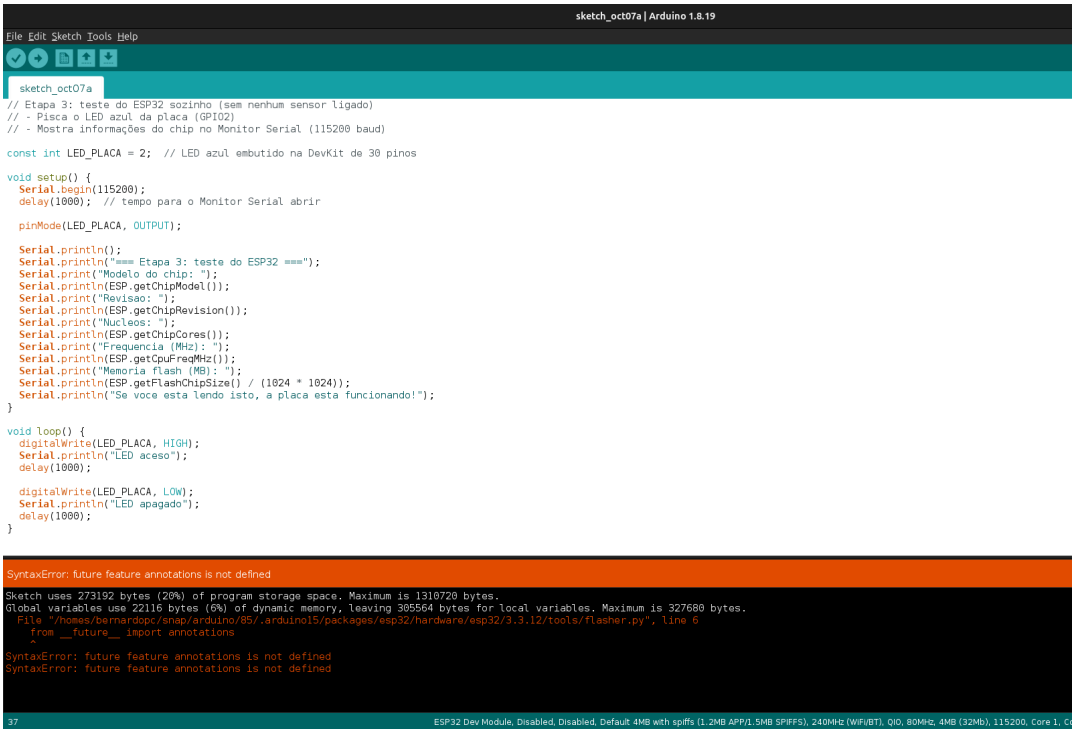
Registrar as dificuldades é parte do método: elas são justamente os pontos em que quem reproduzir o projeto tende a travar. A maioria ocorreu no ambiente de software, e não no hardware.

Tabela 20 – Dificuldades encontradas, causas e soluções.

Sintoma	Causa identificada	Solução adotada
Usuário sem permissão de administrador no computador do laboratório	máquina gerenciada; usuário fora do grupo `dialout`	pedido ao administrador (depois, o usuário recebeu `sudo`)
AppImage da IDE 2 exigia biblioteca extra; ZIP falhou	arquivo ainda não baixado na pasta indicada	IDE 1.8.19 instalada pelo App Center
`SyntaxError: future feature annotations is not defined` ao compilar (Figura 30)	Python antigo embutido no snap da IDE 1.8.19 incompatível com o pacote esp32 3.3.12	remover o pacote e instalar a versão **2.0.17** do pacote esp32
Instalação do pacote parecia travada	download e descompactação longos na IDE 1.8	apagar o pacote parcial e reinstalar, aguardando sem fechar a IDE
IDE indicava porta `/dev/ttyACM0`, esperava-se `/dev/ttyUSB0`	o conversor CH343/CH9102 é CDC-ACM	usar `/dev/ttyACM0`
Erro "port doesn't exist" no Upload	mensagem enganosa: na verdade, falta de permissão na porta	teste com `chmod 666`; depois, regra udev permanente
Permissão perdida ao reconectar a placa	o `chmod` só vale até a desconexão; o grupo só vale após novo login	regra udev `99-esp32.rules` para o ID 1a86:55d4
Erro de compilação "redefinition of setup()"	programa da etapa colado como segunda aba de outro *sketch*	regra: cada etapa em um *sketch* novo (File → New)
Sensor identificado visualmente como BMP280	módulos BMP280 e BME280 são quase idênticos	leitura do registrador 0xD0 (0x60 = BME280) e criação da Etapa 5b
Leituras ocasionais com ERRO no DHT22	mau contato ao mexer nos jumpers	descarte automático de leituras inválidas
Rede do laboratório com login pessoal	ESP32 não suporta WPA2-Enterprise de forma simples	hotspot do celular em 2,4 GHz

Fonte: Elaborado pelos autores.

Figura 30 – Erro real de compilação causado pela incompatibilidade entre a IDE 1.8.19 em snap e o pacote esp32 3.3.12. A compilação do código em si funcionou ("Sketch uses 273192 bytes").



Fonte: Captura de tela do projeto.

> **Lições práticas.** Para quem for reproduzir: (1) use um cabo USB de dados; (2) confira com `lsusb` qual é o chip USB da sua placa, pois ele define o nome da porta; (3) resolva a permissão da porta antes de tudo; (4) use o pacote esp32 2.0.17; (5) nunca cole um programa como aba de outro *sketch*.

8 Limitações e trabalhos futuros

As limitações abaixo referem-se ao protótipo. A estrutura física definitiva (invólucro, abrigo e instalação externa) é um trabalho à parte, fora do escopo deste documento.

Tabela 21 – Limitações atuais e próximos passos do protótipo.

Limitação	Próximo passo proposto
MQ135 reporta tensão, sem calibração	após 24 a 48 h de queima inicial, medir R_L na placa, registrar R_0 em ar externo limpo e exibir R_S/R_0 (e, com ressalvas, ppm equivalente)
Exatidão do PMS7003 não verificada	comparar lado a lado com uma estação oficial próxima e, se necessário, ajustar um fator de correção
Efeito da umidade alta sobre o PM	marcar as leituras de PM quando a umidade do BME280 passar de ~85 %
Histórico só em memória RAM (2 h) e com tempo relativo	obter data e hora por NTP (com internet) ou módulo de relógio DS3231; gravar em cartão microSD
Nome e senha da rede fixos no código	página de configuração da rede, sem regravar o ESP32
Atualização exige cabo USB	atualização do firmware pelo Wi-Fi (OTA)
Travamentos raros não são recuperados	habilitar o *watchdog* para reiniciar o ESP32 automaticamente
Página e rede com senha simples	trocar a senha padrão antes de qualquer uso fora do laboratório

Fonte: Elaborado pelos autores.

9 Conclusão

O protótipo da estação de monitoramento da qualidade do ar foi construído e validado integralmente. Partindo apenas das peças e de um computador de laboratório sem permissões administrativas, chegou-se a um sistema que mede material particulado, gases, temperatura, umidade e pressão; valida e consolida os dados; mostra os resultados em uma página própria acessível pelo celular, sem internet; e publica as séries na nuvem.

A metodologia incremental, com um sensor por etapa, verificações com multímetro antes de cada energização e critérios objetivos de aprovação, permitiu concluir toda a montagem sem nenhum dano a componentes e localizar rapidamente a causa de cada problema. A fundamentação teórica mostrou-se necessária na prática: o divisor de tensão protegeu o ESP32 da saída de 5 V do MQ135; o conhecimento do protocolo I²C revelou que o sensor era um BME280; a validação por soma de verificação garantiu dados íntegros do PMS7003; e a programação não bloqueante permitiu que sensores com ritmos muito diferentes convivessem no mesmo programa.

Os resultados reforçam também as limitações conhecidas dos sensores de baixo custo: eles são excelentes para observar tendências e comparar situações, mas valores absolutos exigem calibração e comparação com instrumentos de referência, especialmente no caso do MQ135. Os próximos passos do protótipo são a calibração do sensor de gases, a validação do PMS7003 e as melhorias de registro de dados e de robustez listadas no Capítulo 8.

Referências

AOSONG ELECTRONICS. **Digital-output relative humidity & temperature sensor/module AM2302 (DHT22)**: datasheet. Guangzhou, [s.d.].

ADAFRUIT INDUSTRIES. **Adafruit BME280 Library**; **Adafruit BMP280 Library**; **DHT sensor library**; **Adafruit Unified Sensor**; **Adafruit BusIO**. Repositórios de código-fonte. Disponível em: https://github.com/adafruit.

ARDUINO. **Arduino IDE**: software. Disponível em: https://www.arduino.cc/en/software.

BOSCH SENSORTEC. **BME280: combined humidity and pressure sensor**: datasheet BST-BME280-DS002. Reutlingen, 2022.

BRASIL. Conselho Nacional do Meio Ambiente. **Resolução CONAMA nº 491, de 19 de novembro de 2018**. Dispõe sobre padrões de qualidade do ar. Diário Oficial da União, Brasília, 21 nov. 2018.

CETESB – COMPANHIA AMBIENTAL DO ESTADO DE SÃO PAULO. **Qualidade do ar**: índice de qualidade do ar. São Paulo. Disponível em: https://cetesb.sp.gov.br/ar/.

ESPRESSIF SYSTEMS. **ESP32 Series Datasheet**. Xangai, 2023a.

ESPRESSIF SYSTEMS. **ESP32 Technical Reference Manual**. Xangai, 2023b.

ESPRESSIF SYSTEMS. **Arduino core for the ESP32** (arduino-esp32), versão 2.0.17. Disponível em: https://github.com/espressif/arduino-esp32.

HANWEI ELECTRONICS. **Technical data MQ-135 gas sensor**: datasheet. Zhengzhou, [s.d.].

KROCKER, G. **MQ135**: Arduino library for the MQ135 air quality sensor. Repositório de código-fonte. Disponível em: https://github.com/GeorgK/MQ135.

MATHWORKS. **ThingSpeak**: documentação e licenças. Disponível em: https://thingspeak.mathworks.com/prices.

NXP SEMICONDUCTORS. **UM10204: I²C-bus specification and user manual**. Rev. 7.0. 2021.

PLANTOWER. **Digital universal particle concentration sensor PMS7003 series**: data manual. Pequim, 2016.

WORLD HEALTH ORGANIZATION. **WHO global air quality guidelines**: particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide. Genebra: WHO, 2021.

Apêndice A – Códigos completos

Todos os programas estão na pasta `estacao/codigo/` do projeto, cada um em sua própria pasta com o mesmo nome do arquivo, como exige a Arduino IDE. Os comentários foram escritos sem acentos para evitar problemas de codificação.

Listagem 9 – Etapa 4: DHT22 (etapa04_dht22.ino).

```
// Etapa 4: teste do DHT22 (temperatura e umidade)
// Ligacao: perna 1 -> 3V3 | perna 2 -> GPIO4 (+ resistor 4,7k ate 3V3) | perna 3 -> nada | perna 4 -> GND
// Biblioteca: "DHT sensor library" (Adafruit) + "Adafruit Unified Sensor"

#include <DHT.h>

const int PINO_DHT = 4;
DHT dht(PINO_DHT, DHT22);

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("=== Etapa 4: teste do DHT22 ===");
  dht.begin();
  delay(2000);  // o DHT22 precisa de ~2 s apos ligar
}

void loop() {
  float umidade = dht.readHumidity();
  float temperatura = dht.readTemperature();  // em graus Celsius

  if (isnan(umidade) || isnan(temperatura)) {
    Serial.println("ERRO: falha ao ler o DHT22 (confira as ligacoes e o resistor)");
  } else {
    Serial.print("Temperatura: ");
    Serial.print(temperatura, 1);
    Serial.print(" C   |   Umidade: ");
    Serial.print(umidade, 1);
    Serial.println(" %");
  }

  delay(2500);  // o DHT22 so aceita uma leitura a cada 2 s
}
```

Listagem 10 – Etapa 5: varredura I²C e identificação do sensor (etapa05_bmp280.ino).

```
// Etapa 5: teste do BMP280 (temperatura e pressao) junto com o DHT22
// Ligacao BMP280: VIN -> 3V3 | GND -> GND | SCL -> GPIO22 | SDA -> GPIO21
// Ligacao DHT22: igual a Etapa 4 (dados no GPIO4)
// Bibliotecas: "Adafruit BMP280 Library" (+ "Adafruit BusIO") e "DHT sensor library"

#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <DHT.h>

const int PINO_DHT = 4;
const int PINO_SDA = 21;
const int PINO_SCL = 22;

DHT dht(PINO_DHT, DHT22);
Adafruit_BMP280 bmp;
bool bmpOk = false;

```

```
// Le o registrador de identificacao (0xD0) do sensor no endereco I2C informado.
// 0x58 = BMP280, 0x60 = BME280, 0 = nao respondeu
uint8_t lerChipId(uint8_t endereco) {
  Wire.beginTransmission(endereco);
  Wire.write(0xD0);
  if (Wire.endTransmission() != 0) return 0;
  Wire.requestFrom((int)endereco, 1);
  if (Wire.available()) return Wire.read();
  return 0;
}

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("=== Etapa 5: teste do BMP280 + DHT22 ===");

  Wire.begin(PINO_SDA, PINO_SCL);

  // 1) Procura dispositivos no barramento I2C
  Serial.println("Procurando dispositivos I2C...");
  int encontrados = 0;
  for (uint8_t endereco = 1; endereco < 127; endereco++) {
    Wire.beginTransmission(endereco);
    if (Wire.endTransmission() == 0) {
      Serial.print("  Dispositivo encontrado no endereco 0x");
      Serial.println(endereco, HEX);
      encontrados++;
    }
  }
  if (encontrados == 0) {
    Serial.println("  NENHUM dispositivo I2C encontrado! Confira SDA (D21), SCL (D22), VIN e GND.");
  }

  // 2) Identifica o chip e inicia o sensor
  const uint8_t enderecos[] = {0x76, 0x77};
  for (uint8_t i = 0; i < 2; i++) {
    uint8_t id = lerChipId(enderecos[i]);
    if (id == 0) continue;
    Serial.print("Chip no endereco 0x");
    Serial.print(enderecos[i], HEX);
    Serial.print(" tem ID 0x");
    Serial.print(id, HEX);
    if (id == 0x58) Serial.println(" -> e um BMP280");
    else if (id == 0x60) Serial.println(" -> e um BME280 (tambem mede umidade!)");
    else Serial.println(" -> ID desconhecido");

    if (bmp.begin(enderecos[i], id)) {
      bmpOk = true;
      break;
    }
  }
  if (!bmpOk) {
    Serial.println("ERRO: BMP280 nao iniciou. Confira as ligacoes.");
  }

  dht.begin();
  delay(2000);
}

void loop() {
  Serial.println("----------------------------------------");

  if (bmpOk) {
    Serial.print("BMP280  Temperatura: ");
    Serial.print(bmp.readTemperature(), 1);
```

```
    Serial.print(" C   |   Pressao: ");
    Serial.print(bmp.readPressure() / 100.0F, 1);
    Serial.println(" hPa");
  } else {
    Serial.println("BMP280  (sem leitura)");
  }

  float umidade = dht.readHumidity();
  float temperatura = dht.readTemperature();
  if (isnan(umidade) || isnan(temperatura)) {
    Serial.println("DHT22   ERRO de leitura");
  } else {
    Serial.print("DHT22   Temperatura: ");
    Serial.print(temperatura, 1);
    Serial.print(" C   |   Umidade: ");
    Serial.print(umidade, 1);
    Serial.println(" %");
  }

  delay(2500);
}
```

Listagem 11 – Etapa 5b: BME280 com umidade + DHT22 (etapa05b_bme280.ino).

```
// Etapa 5b: o sensor e um BME280 (ID 0x60) -> agora lendo tambem a umidade dele
// Ligacao: igual a Etapa 5 (VIN -> 3V3 | GND -> GND | SCL -> GPIO22 | SDA -> GPIO21) + DHT22 no GPIO4
// Bibliotecas: "Adafruit BME280 Library" (+ "Adafruit Unified Sensor", "Adafruit BusIO") e "DHT sensor library"

#include <Wire.h>
#include <Adafruit_BME280.h>
#include <DHT.h>

const int PINO_DHT = 4;
const int PINO_SDA = 21;
const int PINO_SCL = 22;

DHT dht(PINO_DHT, DHT22);
Adafruit_BME280 bme;
bool bmeOk = false;

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("=== Etapa 5b: teste do BME280 + DHT22 ===");

  Wire.begin(PINO_SDA, PINO_SCL);

  if (bme.begin(0x76, &Wire)) {
    bmeOk = true;
    Serial.println("BME280 iniciado no endereco 0x76");
  } else if (bme.begin(0x77, &Wire)) {
    bmeOk = true;
    Serial.println("BME280 iniciado no endereco 0x77");
  } else {
    Serial.println("ERRO: BME280 nao iniciou. Confira as ligacoes.");
  }

  dht.begin();
  delay(2000);
}

void loop() {
  Serial.println("------------------------------------------------------------");
```

```
41
42  if (bmeOk) {
43    Serial.print("BME280  Temperatura: ");
44    Serial.print(bme.readTemperature(), 1);
45    Serial.print(" C  |  Umidade: ");
46    Serial.print(bme.readHumidity(), 1);
47    Serial.print(" %  |  Pressao: ");
48    Serial.print(bme.readPressure() / 100.0F, 1);
49    Serial.println(" hPa");
50  } else {
51    Serial.println("BME280  (sem leitura)");
52  }
53
54  float umidade = dht.readHumidity();
55  float temperatura = dht.readTemperature();
56  if (isnan(umidade) || isnan(temperatura)) {
57    Serial.println("DHT22   ERRO de leitura");
58  } else {
59    Serial.print("DHT22   Temperatura: ");
60    Serial.print(temperatura, 1);
61    Serial.print(" C  |  Umidade: ");
62    Serial.print(umidade, 1);
63    Serial.println(" %");
64  }
65
66  delay(2500);
67 }
```

Listagem 12 – Etapa 6: PMS7003 (etapa06_pms7003.ino).

```
1  // Etapa 6: teste do PMS7003 (material particulado PM1.0 / PM2.5 / PM10)
2  // Ligacao (placa adaptadora, conector P3): TX -> GPIO16 | RX -> GPIO17 | VCC -> VIN (5 V) |
   GND -> GND
3  // Nao precisa de biblioteca: o pacote de dados do sensor e lido e conferido aqui mesmo.
4
5  const int PMS_RX = 16;  // recebe o TX do PMS7003
6  const int PMS_TX = 17;  // envia para o RX do PMS7003
7
8  struct PmsDados {
9    uint16_t pm1_0;   // ug/m3 (atmosferico)
10   uint16_t pm2_5;   // ug/m3 (atmosferico)
11   uint16_t pm10;    // ug/m3 (atmosferico)
12   uint16_t n0_3;    // particulas > 0,3 um em 0,1 L de ar
13   uint16_t n2_5;    // particulas > 2,5 um em 0,1 L de ar
14 };
15
16 uint8_t buf[32];
17 int idx = 0;
18 unsigned long pacotesOk = 0;
19 unsigned long pacotesRuins = 0;
20
21 // Le os bytes que chegaram do sensor. Devolve true quando um pacote completo e valido foi
   lido.
22 // Pacote do PMS7003: 32 bytes, comeca com 0x42 0x4D, termina com a soma de verificacao.
23 bool lerPms(PmsDados &d) {
24   while (Serial2.available()) {
25     uint8_t b = Serial2.read();
26
27     if (idx == 0) {
28       if (b == 0x42) buf[idx++] = b;
29       continue;
30     }
31     if (idx == 1 && b != 0x4D) {
32       idx = (b == 0x42) ? 1 : 0;
33       continue;
34     }
```

```
35
36      buf[idx++] = b;
37      if (idx < 32) continue;
38      idx = 0;
39
40      uint16_t comprimento = (buf[2] << 8) | buf[3];
41      uint16_t soma = 0;
42      for (int i = 0; i < 30; i++) soma += buf[i];
43      uint16_t verificacao = (buf[30] << 8) | buf[31];
44
45      if (comprimento != 28 || soma != verificacao) {
46        pacotesRuins++;
47        continue;
48      }
49
50      d.pm1_0 = (buf[10] << 8) | buf[11];
51      d.pm2_5 = (buf[12] << 8) | buf[13];
52      d.pm10  = (buf[14] << 8) | buf[15];
53      d.n0_3  = (buf[16] << 8) | buf[17];
54      d.n2_5  = (buf[22] << 8) | buf[23];
55      pacotesOk++;
56      return true;
57    }
58    return false;
59  }
60
61  void setup() {
62    Serial.begin(115200);
63    delay(1000);
64    Serial.println();
65    Serial.println("=== Etapa 6: teste do PMS7003 ===");
66    Serial2.begin(9600, SERIAL_8N1, PMS_RX, PMS_TX);
67    Serial.println("Aguardando dados do sensor (as leituras estabilizam depois de ~30 s)...");
68  }
69
70  unsigned long ultimoDado = 0;
71
72  void loop() {
73    PmsDados d;
74    if (lerPms(d)) {
75      ultimoDado = millis();
76      Serial.print("PM1.0: ");
77      Serial.print(d.pm1_0);
78      Serial.print("  |  PM2.5: ");
79      Serial.print(d.pm2_5);
80      Serial.print("  |  PM10: ");
81      Serial.print(d.pm10);
82      Serial.print(" ug/m3   |  >0,3um: ");
83      Serial.print(d.n0_3);
84      Serial.print("  >2,5um: ");
85      Serial.print(d.n2_5);
86      Serial.print("   [ok: ");
87      Serial.print(pacotesOk);
88      Serial.print(" / ruins: ");
89      Serial.print(pacotesRuins);
90      Serial.println("]");
91    }
92
93    if (millis() - ultimoDado > 5000) {
94      Serial.println("Nenhum dado do PMS7003 ha 5 s: confira TX->GPIO16, VCC no VIN e se a
    ventoinha esta girando.");
95      ultimoDado = millis();
96    }
97  }
```

Listagem 13 – Etapa 7: MQ135 (etapa07_mq135.ino).

```
// Etapa 7: teste do MQ135 (gases / qualidade do ar)
// Ligacao: VCC -> VIN (5 V) | GND -> GND | DO -> nada
//          AO -> resistor 4,7k -> ponto do meio -> resistor 4,7k -> GND
//          ponto do meio -> GPIO34
// O divisor reduz a tensao pela metade: o ESP32 nunca recebe mais que 2,5 V.

const int PINO_MQ135 = 34;
const float FATOR_DIVISOR = 2.0;  // dois resistores iguais: tensao no AO = 2 x tensao no pino
const int AMOSTRAS = 20;          // media de varias leituras para reduzir ruido

unsigned long inicio;

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("=== Etapa 7: teste do MQ135 ===");
  Serial.println("O sensor precisa aquecer: as leituras estabilizam depois de alguns minutos.");

  analogReadResolution(12);                          // 0 a 4095
  analogSetPinAttenuation(PINO_MQ135, ADC_11db);   // faixa de ~0 a 3,1 V
  inicio = millis();
}

void loop() {
  uint32_t somaMv = 0;
  uint32_t somaBruto = 0;
  for (int i = 0; i < AMOSTRAS; i++) {
    somaMv += analogReadMilliVolts(PINO_MQ135);
    somaBruto += analogRead(PINO_MQ135);
    delay(10);
  }
  float mvPino = somaMv / (float)AMOSTRAS;
  int bruto = somaBruto / AMOSTRAS;
  float vSaidaAO = mvPino * FATOR_DIVISOR / 1000.0;
  unsigned long segundos = (millis() - inicio) / 1000;

  Serial.print("Tempo: ");
  Serial.print(segundos);
  Serial.print(" s  |  Leitura bruta: ");
  Serial.print(bruto);
  Serial.print("  |  GPIO34: ");
  Serial.print(mvPino, 0);
  Serial.print(" mV  |  Saida AO do MQ135: ");
  Serial.print(vSaidaAO, 2);
  Serial.println(" V");

  if (mvPino > 2800) {
    Serial.println("ATENCAO: tensao no GPIO34 acima do esperado. Desconecte o USB e confira o divisor!");
  }

  delay(2000);
}
```

Listagem 14 – Etapa 8: estação integrada (etapa08_estacao_integrada.ino).

```
// Etapa 8: estacao integrada - todos os sensores juntos
// ESP32 DevKit 30 pinos | core esp32 2.0.17
//
// Ligacoes (iguais as etapas anteriores):
//   BME280 : VIN -> 3V3 | GND -> GND | SDA -> GPIO21 | SCL -> GPIO22
//   DHT22  : perna 1 -> 3V3 | perna 2 -> GPIO4 (+ 4,7k ate 3V3) | perna 4 -> GND
```

```
//   PMS7003: TX -> GPIO16 | RX -> GPIO17 | VCC -> VIN | GND -> GND
//   MQ135  : VCC -> VIN | GND -> GND | AO -> divisor 4,7k/4,7k -> GPIO34
//
// Bibliotecas: "Adafruit BME280 Library" e "DHT sensor library" (Adafruit)

#include <Wire.h>
#include <Adafruit_BME280.h>
#include <DHT.h>

// ---------------- Pinos ----------------
const int PINO_SDA   = 21;
const int PINO_SCL   = 22;
const int PINO_DHT   = 4;
const int PMS_RX     = 16;  // recebe o TX do PMS7003
const int PMS_TX     = 17;  // envia para o RX do PMS7003
const int PINO_MQ135 = 34;

// ---------------- Configuracoes ----------------
const unsigned long INTERVALO_RELATORIO = 5000;    // imprime um resumo a cada 5 s
const unsigned long INTERVALO_DHT       = 2500;    // o DHT22 aceita 1 leitura a cada 2 s
const unsigned long INTERVALO_MQ        = 100;     // amostra do MQ135 a cada 100 ms
const unsigned long VALIDADE_DHT        = 10000;   // leitura do DHT22 vale por 10 s
const unsigned long VALIDADE_PMS        = 5000;    // sem pacote do PMS por 5 s = falha
const unsigned long AQUECIMENTO_MQ      = 180000;  // MQ135 aquecendo nos primeiros 3 min
const float FATOR_DIVISOR_MQ            = 2.0;     // dois resistores iguais de 4,7k

// ---------------- Objetos ----------------
Adafruit_BME280 bme;
DHT dht(PINO_DHT, DHT22);

// ---------------- Estado dos sensores ----------------
bool bmeOk = false;

float dhtTemp = NAN, dhtUmid = NAN;
unsigned long dhtUltimaLeitura = 0;   // momento da ultima leitura VALIDA
unsigned long dhtUltimaTentativa = 0;
unsigned long dhtFalhas = 0;

uint8_t pmsBuf[32];
int pmsIdx = 0;
unsigned long pmsUltimoPacote = 0;
unsigned long pmsPacotesOk = 0, pmsPacotesRuins = 0;
uint32_t pmsSoma1 = 0, pmsSoma25 = 0, pmsSoma10 = 0, pmsQtd = 0;   // acumulado do periodo
uint16_t pms1 = 0, pms25 = 0, pms10 = 0;                           // media do ultimo periodo
bool pmsTemDado = false;

uint32_t mqSomaMv = 0, mqQtd = 0;
unsigned long mqUltimaAmostra = 0;

unsigned long ultimoRelatorio = 0;
unsigned long numeroLeitura = 0;

// ========================================================================
// PMS7003: le os bytes recebidos e acumula cada pacote valido
// Pacote: 32 bytes, comeca com 0x42 0x4D, termina com soma de verificacao
// ========================================================================
void processarPms() {
  while (Serial2.available()) {
    uint8_t b = Serial2.read();

    if (pmsIdx == 0) {
      if (b == 0x42) pmsBuf[pmsIdx++] = b;
      continue;
    }
    if (pmsIdx == 1 && b != 0x4D) {
      pmsIdx = (b == 0x42) ? 1 : 0;
      continue;
```

```
    }

    pmsBuf[pmsIdx++] = b;
    if (pmsIdx < 32) continue;
    pmsIdx = 0;

    uint16_t comprimento = (pmsBuf[2] << 8) | pmsBuf[3];
    uint16_t soma = 0;
    for (int i = 0; i < 30; i++) soma += pmsBuf[i];
    uint16_t verificacao = (pmsBuf[30] << 8) | pmsBuf[31];
    if (comprimento != 28 || soma != verificacao) {
      pmsPacotesRuins++;
      continue;
    }

    pmsSoma1  += (pmsBuf[10] << 8) | pmsBuf[11];
    pmsSoma25 += (pmsBuf[12] << 8) | pmsBuf[13];
    pmsSoma10 += (pmsBuf[14] << 8) | pmsBuf[15];
    pmsQtd++;
    pmsPacotesOk++;
    pmsUltimoPacote = millis();
  }
}

// ============================================================================
// DHT22: tenta ler a cada 2,5 s e guarda a ultima leitura valida
// ============================================================================
void processarDht() {
  if (millis() - dhtUltimaTentativa < INTERVALO_DHT) return;
  dhtUltimaTentativa = millis();

  float u = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(u) || isnan(t)) {
    dhtFalhas++;
    return;
  }
  dhtUmid = u;
  dhtTemp = t;
  dhtUltimaLeitura = millis();
}

// ============================================================================
// MQ135: acumula amostras para tirar a media no relatorio
// ============================================================================
void processarMq() {
  if (millis() - mqUltimaAmostra < INTERVALO_MQ) return;
  mqUltimaAmostra = millis();
  mqSomaMv += analogReadMilliVolts(PINO_MQ135);
  mqQtd++;
}

// ============================================================================
// Classificacao do PM2.5 (faixas de referencia para media de 24 h,
// usadas aqui so como indicacao para a leitura instantanea)
// ============================================================================
const char* classificarPm25(uint16_t pm) {
  if (pm <= 25)  return "BOA";
  if (pm <= 50)  return "MODERADA";
  if (pm <= 75)  return "RUIM";
  if (pm <= 125) return "MUITO RUIM";
  return "PESSIMA";
}

void imprimirTempoLigado() {
  unsigned long s = millis() / 1000;
  char texto[24];
```

```
  snprintf(texto, sizeof(texto), "%02lu:%02lu:%02lu", s / 3600, (s / 60) % 60, s % 60);
  Serial.print(texto);
}

// ================================================================
// Relatorio a cada 5 s
// ================================================================
void imprimirRelatorio() {
  numeroLeitura++;

  // fecha o periodo do PMS7003
  if (pmsQtd > 0) {
    pms1  = pmsSoma1  / pmsQtd;
    pms25 = pmsSoma25 / pmsQtd;
    pms10 = pmsSoma10 / pmsQtd;
    pmsTemDado = true;
  }
  uint32_t pmsQtdPeriodo = pmsQtd;
  pmsSoma1 = pmsSoma25 = pmsSoma10 = pmsQtd = 0;
  bool pmsOk = pmsTemDado && (millis() - pmsUltimoPacote < VALIDADE_PMS);

  // fecha o periodo do MQ135
  float mqMv = (mqQtd > 0) ? (float)mqSomaMv / mqQtd : NAN;
  mqSomaMv = mqQtd = 0;
  float mqVolts = mqMv * FATOR_DIVISOR_MQ / 1000.0;
  bool mqOk = !isnan(mqMv) && mqMv > 20 && mqMv < 2800;

  bool dhtOk = dhtUltimaLeitura > 0 && (millis() - dhtUltimaLeitura < VALIDADE_DHT);

  Serial.println();
  Serial.print("===== Leitura #");
  Serial.print(numeroLeitura);
  Serial.print("   (ligado ha ");
  imprimirTempoLigado();
  Serial.println(") =====");

  // Temperatura e umidade
  Serial.print("Temperatura : ");
  if (bmeOk) { Serial.print(bme.readTemperature(), 1); Serial.print(" C (BME280)"); }
  else Serial.print("--- (BME280)");
  Serial.print("  |  ");
  if (dhtOk) { Serial.print(dhtTemp, 1); Serial.println(" C (DHT22)"); }
  else Serial.println("--- (DHT22)");

  Serial.print("Umidade     : ");
  if (bmeOk) { Serial.print(bme.readHumidity(), 1); Serial.print(" % (BME280)"); }
  else Serial.print("--- (BME280)");
  Serial.print("  |  ");
  if (dhtOk) { Serial.print(dhtUmid, 1); Serial.println(" % (DHT22)"); }
  else Serial.println("--- (DHT22)");

  Serial.print("Pressao     : ");
  if (bmeOk) { Serial.print(bme.readPressure() / 100.0F, 1); Serial.println(" hPa"); }
  else Serial.println("---");

  // Particulas
  Serial.print("Particulas  : ");
  if (pmsOk) {
    Serial.print("PM1.0 = ");  Serial.print(pms1);
    Serial.print("  |  PM2.5 = "); Serial.print(pms25);
    Serial.print("  |  PM10 = ");  Serial.print(pms10);
    Serial.print(" ug/m3  (media de ");
    Serial.print(pmsQtdPeriodo);
    Serial.println(" pacotes)");
    Serial.print("Qualidade   : ");
    Serial.print(classificarPm25(pms25));
    Serial.println(" (pelo PM2.5)");
```

```
  } else {
    Serial.println("--- (sem dados do PMS7003)");
  }

  // Gases
  Serial.print("Gases MQ135 : ");
  if (mqOk) {
    Serial.print(mqVolts, 2);
    Serial.print(" V");
    if (millis() < AQUECIMENTO_MQ) Serial.print("  [aquecendo, valor ainda instavel]");
    Serial.println();
  } else {
    Serial.println("--- (leitura fora da faixa, confira o divisor)");
  }

  // Situacao de cada sensor
  Serial.print("Status      : BME280 ");
  Serial.print(bmeOk ? "ok" : "FALHA");
  Serial.print("  |  DHT22 ");
  Serial.print(dhtOk ? "ok" : "FALHA");
  Serial.print(" (");
  Serial.print(dhtFalhas);
  Serial.print(" falhas)  |  PMS7003 ");
  Serial.print(pmsOk ? "ok" : "FALHA");
  Serial.print(" (");
  Serial.print(pmsPacotesRuins);
  Serial.print(" pacotes ruins)  |  MQ135 ");
  Serial.println(mqOk ? "ok" : "FALHA");
}

// ================================================================
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("=== Estacao de Qualidade do Ar - Etapa 8 (integrada) ===");

  // BME280
  Wire.begin(PINO_SDA, PINO_SCL);
  if (bme.begin(0x76, &Wire) || bme.begin(0x77, &Wire)) {
    bmeOk = true;
    Serial.println("BME280  : iniciado");
  } else {
    Serial.println("BME280  : NAO encontrado (confira SDA/SCL/VIN/GND)");
  }

  // DHT22
  dht.begin();
  Serial.println("DHT22   : iniciado");

  // PMS7003
  Serial2.begin(9600, SERIAL_8N1, PMS_RX, PMS_TX);
  Serial.println("PMS7003 : porta serial iniciada");

  // MQ135
  analogReadResolution(12);
  analogSetPinAttenuation(PINO_MQ135, ADC_11db);
  Serial.println("MQ135   : entrada analogica iniciada");

  Serial.println("Primeiro resumo em 5 segundos...");
  ultimoRelatorio = millis();
}

void loop() {
  processarPms();
  processarDht();
  processarMq();
```

```
275
276   if (millis() - ultimoRelatorio >= INTERVALO_RELATORIO) {
277     ultimoRelatorio = millis();
278     imprimirRelatorio();
279   }
280 }
```

Listagem 15 – Etapa 9: programa final (estacao_final.ino).

```
  1 // ====================================================================
  2 // Estacao de Monitoramento da Qualidade do Ar - VERSAO FINAL
  3 // ESP32 DevKit 30 pinos | core esp32 2.0.17
  4 //
  5 // O que faz:
  6 //  - Le BME280, DHT22, PMS7003 e MQ135 (igual a Etapa 8)
  7 //  - Cria a propria rede Wi-Fi "EstacaoAr": conecte o celular/PC e abra
  8 //    http://192.168.4.1 para ver os valores e os graficos (funciona sem internet)
  9 //  - Se houver uma rede com internet configurada, envia os dados ao ThingSpeak
 10 //  - Continua imprimindo o resumo no Monitor Serial (115200)
 11 //
 12 // Ligacoes:
 13 //   BME280 : VIN -> 3V3 | GND -> GND | SDA -> GPIO21 | SCL -> GPIO22
 14 //   DHT22  : perna 1 -> 3V3 | perna 2 -> GPIO4 (+ 4,7k ate 3V3) | perna 4 -> GND
 15 //   PMS7003: TX -> GPIO16 | RX -> GPIO17 | VCC -> VIN | GND -> GND
 16 //   MQ135  : VCC -> VIN | GND -> GND | AO -> divisor 4,7k/4,7k -> GPIO34
 17 //
 18 // Bibliotecas: "Adafruit BME280 Library" e "DHT sensor library" (Adafruit)
 19 // ====================================================================
 20
 21 #include <Wire.h>
 22 #include <Adafruit_BME280.h>
 23 #include <DHT.h>
 24 #include <WiFi.h>
 25 #include <WebServer.h>
 26 #include <HTTPClient.h>
 27
 28 // ====================================================================
 29 // CONFIGURACAO - edite somente esta parte
 30 // ====================================================================
 31 // Rede propria da estacao (sempre ligada)
 32 const char* AP_NOME  = "EstacaoAr";
 33 const char* AP_SENHA = "qualidade123";      // minimo 8 caracteres
 34
 35 // Rede com internet para enviar ao ThingSpeak (deixe "" para nao usar)
 36 const char* WIFI_NOME  = "";
 37 const char* WIFI_SENHA = "";
 38
 39 // ThingSpeak: "Write API Key" do seu canal (deixe "" para nao enviar)
 40 const char* THINGSPEAK_CHAVE = "";
 41 // ====================================================================
 42
 43 // ---------------- Pinos ----------------
 44 const int PINO_SDA   = 21;
 45 const int PINO_SCL   = 22;
 46 const int PINO_DHT   = 4;
 47 const int PMS_RX     = 16;
 48 const int PMS_TX     = 17;
 49 const int PINO_MQ135 = 34;
 50
 51 // ---------------- Tempos ----------------
 52 const unsigned long INTERVALO_RELATORIO  = 5000;     // atualiza leituras a cada 5 s
 53 const unsigned long INTERVALO_DHT        = 2500;
 54 const unsigned long INTERVALO_MQ         = 100;
 55 const unsigned long VALIDADE_DHT         = 10000;
 56 const unsigned long VALIDADE_PMS         = 5000;
 57 const unsigned long AQUECIMENTO_MQ       = 180000;
```

```
const unsigned long INTERVALO_HISTORICO  = 30000;    // um ponto no grafico a cada 30 s
const unsigned long INTERVALO_THINGSPEAK = 20000;    // o plano gratuito aceita no minimo 15 s
const unsigned long INTERVALO_RECONEXAO  = 30000;
const float FATOR_DIVISOR_MQ             = 2.0;

// ---------------- Objetos ----------------
Adafruit_BME280 bme;
DHT dht(PINO_DHT, DHT22);
WebServer servidor(80);

// ---------------- Leituras atuais ----------------
struct Leituras {
  float tBme = NAN, uBme = NAN, pressao = NAN;
  float tDht = NAN, uDht = NAN;
  float mqVolts = NAN;
  int pm1 = -1, pm25 = -1, pm10 = -1;
  bool bmeOk = false, dhtOk = false, pmsOk = false, mqOk = false;
  bool mqAquecendo = true;
};
Leituras atual;

// ---------------- Estado interno ----------------
bool bmeIniciado = false;

float dhtTemp = NAN, dhtUmid = NAN;
unsigned long dhtUltimaLeitura = 0, dhtUltimaTentativa = 0, dhtFalhas = 0;

uint8_t pmsBuf[32];
int pmsIdx = 0;
unsigned long pmsUltimoPacote = 0, pmsPacotesOk = 0, pmsPacotesRuins = 0;
uint32_t pmsSoma1 = 0, pmsSoma25 = 0, pmsSoma10 = 0, pmsQtd = 0;
bool pmsTemDado = false;
int pmsUlt1 = 0, pmsUlt25 = 0, pmsUlt10 = 0;

uint32_t mqSomaMv = 0, mqQtd = 0;
unsigned long mqUltimaAmostra = 0;

unsigned long ultimoRelatorio = 0, numeroLeitura = 0;
unsigned long ultimoHistorico = 0, ultimoEnvio = 0, ultimaReconexao = 0;
unsigned long envioOk = 0, envioFalhas = 0;
String ultimoErroEnvio = "";

// ---------------- Historico (para os graficos) ----------------
const int TAM_HISTORICO = 240;   // 240 pontos x 30 s = 2 horas
struct Ponto {
  uint32_t seg;
  float pm25, tBme, uBme, mq;
};
Ponto historico[TAM_HISTORICO];
int histInicio = 0, histQtd = 0;

void guardarHistorico() {
  int pos = (histInicio + histQtd) % TAM_HISTORICO;
  historico[pos].seg  = millis() / 1000;
  historico[pos].pm25 = atual.pmsOk ? atual.pm25 : NAN;
  historico[pos].tBme = atual.bmeOk ? atual.tBme : NAN;
  historico[pos].uBme = atual.bmeOk ? atual.uBme : NAN;
  historico[pos].mq   = atual.mqOk  ? atual.mqVolts : NAN;
  if (histQtd < TAM_HISTORICO) histQtd++;
  else histInicio = (histInicio + 1) % TAM_HISTORICO;
}

// ========================================================================
// Leitura dos sensores
// ========================================================================
void processarPms() {
  while (Serial2.available()) {
```

```
    uint8_t b = Serial2.read();
    if (pmsIdx == 0) {
      if (b == 0x42) pmsBuf[pmsIdx++] = b;
      continue;
    }
    if (pmsIdx == 1 && b != 0x4D) {
      pmsIdx = (b == 0x42) ? 1 : 0;
      continue;
    }
    pmsBuf[pmsIdx++] = b;
    if (pmsIdx < 32) continue;
    pmsIdx = 0;

    uint16_t comprimento = (pmsBuf[2] << 8) | pmsBuf[3];
    uint16_t soma = 0;
    for (int i = 0; i < 30; i++) soma += pmsBuf[i];
    uint16_t verificacao = (pmsBuf[30] << 8) | pmsBuf[31];
    if (comprimento != 28 || soma != verificacao) {
      pmsPacotesRuins++;
      continue;
    }
    pmsSoma1  += (pmsBuf[10] << 8) | pmsBuf[11];
    pmsSoma25 += (pmsBuf[12] << 8) | pmsBuf[13];
    pmsSoma10 += (pmsBuf[14] << 8) | pmsBuf[15];
    pmsQtd++;
    pmsPacotesOk++;
    pmsUltimoPacote = millis();
  }
}

void processarDht() {
  if (millis() - dhtUltimaTentativa < INTERVALO_DHT) return;
  dhtUltimaTentativa = millis();
  float u = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(u) || isnan(t)) {
    dhtFalhas++;
    return;
  }
  dhtUmid = u;
  dhtTemp = t;
  dhtUltimaLeitura = millis();
}

void processarMq() {
  if (millis() - mqUltimaAmostra < INTERVALO_MQ) return;
  mqUltimaAmostra = millis();
  mqSomaMv += analogReadMilliVolts(PINO_MQ135);
  mqQtd++;
}

const char* classificarPm25(int pm) {
  if (pm < 0)    return "---";
  if (pm <= 25)  return "BOA";
  if (pm <= 50)  return "MODERADA";
  if (pm <= 75)  return "RUIM";
  if (pm <= 125) return "MUITO RUIM";
  return "PESSIMA";
}

// Fecha o periodo de 5 s e atualiza "atual"
void atualizarLeituras() {
  // BME280
  atual.bmeOk = bmeIniciado;
  if (bmeIniciado) {
    atual.tBme = bme.readTemperature();
    atual.uBme = bme.readHumidity();
```

```
    atual.pressao = bme.readPressure() / 100.0F;
    if (isnan(atual.tBme) || isnan(atual.pressao) || atual.pressao < 300 || atual.pressao > 1100) {
      atual.bmeOk = false;
    }
  }

  // DHT22
  atual.dhtOk = dhtUltimaLeitura > 0 && (millis() - dhtUltimaLeitura < VALIDADE_DHT);
  atual.tDht = dhtTemp;
  atual.uDht = dhtUmid;

  // PMS7003 (media do periodo)
  if (pmsQtd > 0) {
    pmsUlt1  = pmsSoma1  / pmsQtd;
    pmsUlt25 = pmsSoma25 / pmsQtd;
    pmsUlt10 = pmsSoma10 / pmsQtd;
    pmsTemDado = true;
  }
  pmsSoma1 = pmsSoma25 = pmsSoma10 = pmsQtd = 0;
  atual.pmsOk = pmsTemDado && (millis() - pmsUltimoPacote < VALIDADE_PMS);
  atual.pm1 = pmsUlt1;
  atual.pm25 = pmsUlt25;
  atual.pm10 = pmsUlt10;

  // MQ135 (media do periodo)
  float mqMv = (mqQtd > 0) ? (float)mqSomaMv / mqQtd : NAN;
  mqSomaMv = mqQtd = 0;
  atual.mqOk = !isnan(mqMv) && mqMv > 20 && mqMv < 2800;
  atual.mqVolts = mqMv * FATOR_DIVISOR_MQ / 1000.0;
  atual.mqAquecendo = millis() < AQUECIMENTO_MQ;
}

// ============================================================================
// Monitor Serial
// ============================================================================
void imprimirTempo(unsigned long s) {
  char texto[24];
  snprintf(texto, sizeof(texto), "%02lu:%02lu:%02lu", s / 3600, (s / 60) % 60, s % 60);
  Serial.print(texto);
}

void imprimirRelatorio() {
  numeroLeitura++;
  Serial.println();
  Serial.print("===== Leitura #");
  Serial.print(numeroLeitura);
  Serial.print("   (ligado ha ");
  imprimirTempo(millis() / 1000);
  Serial.println(") =====");

  Serial.print("Temperatura : ");
  if (atual.bmeOk) { Serial.print(atual.tBme, 1); Serial.print(" C (BME280)"); } else Serial.print("--- (BME280)");
  Serial.print("  |  ");
  if (atual.dhtOk) { Serial.print(atual.tDht, 1); Serial.println(" C (DHT22)"); } else Serial.println("--- (DHT22)");

  Serial.print("Umidade     : ");
  if (atual.bmeOk) { Serial.print(atual.uBme, 1); Serial.print(" % (BME280)"); } else Serial.print("--- (BME280)");
  Serial.print("  |  ");
  if (atual.dhtOk) { Serial.print(atual.uDht, 1); Serial.println(" % (DHT22)"); } else Serial.println("--- (DHT22)");

  Serial.print("Pressao     : ");
```

```
  if (atual.bmeOk) { Serial.print(atual.pressao, 1); Serial.println(" hPa"); } else
Serial.println("---");

  Serial.print("Particulas  : ");
  if (atual.pmsOk) {
    Serial.print("PM1.0 = ");  Serial.print(atual.pm1);
    Serial.print("  |  PM2.5 = "); Serial.print(atual.pm25);
    Serial.print("  |  PM10 = ");  Serial.print(atual.pm10);
    Serial.println(" ug/m3");
    Serial.print("Qualidade   : ");
    Serial.print(classificarPm25(atual.pm25));
    Serial.println(" (pelo PM2.5)");
  } else {
    Serial.println("--- (sem dados do PMS7003)");
  }

  Serial.print("Gases MQ135 : ");
  if (atual.mqOk) {
    Serial.print(atual.mqVolts, 2);
    Serial.print(" V");
    if (atual.mqAquecendo) Serial.print("  [aquecendo, valor ainda instavel]");
    Serial.println();
  } else {
    Serial.println("--- (leitura fora da faixa, confira o divisor)");
  }

  Serial.print("Status      : BME280 ");
  Serial.print(atual.bmeOk ? "ok" : "FALHA");
  Serial.print("  |  DHT22 ");
  Serial.print(atual.dhtOk ? "ok" : "FALHA");
  Serial.print("  |  PMS7003 ");
  Serial.print(atual.pmsOk ? "ok" : "FALHA");
  Serial.print("  |  MQ135 ");
  Serial.println(atual.mqOk ? "ok" : "FALHA");

  Serial.print("Rede        : pagina em http://");
  Serial.print(WiFi.softAPIP().toString());
  Serial.print("  |  internet: ");
  if (strlen(WIFI_NOME) == 0) Serial.print("nao configurada");
  else if (WiFi.status() == WL_CONNECTED) Serial.print("conectada");
  else Serial.print("desconectada");
  Serial.print("  |  ThingSpeak: ");
  if (strlen(THINGSPEAK_CHAVE) == 0) Serial.println("nao configurado");
  else {
    Serial.print(envioOk);
    Serial.print(" envios ok, ");
    Serial.print(envioFalhas);
    Serial.println(" falhas");
  }
}

// ============================================================================
// ThingSpeak
// Campos: 1 temp BME | 2 umid BME | 3 pressao | 4 PM1.0 | 5 PM2.5
//         6 PM10 | 7 MQ135 (V) | 8 umid DHT22
// ============================================================================
void adicionarCampo(String &url, int campo, float valor, bool valido, int casas) {
  if (!valido || isnan(valor)) return;
  url += "&field";
  url += campo;
  url += "=";
  url += String(valor, casas);
}

void enviarThingSpeak() {
  if (strlen(THINGSPEAK_CHAVE) == 0 || strlen(WIFI_NOME) == 0) return;
  if (millis() - ultimoEnvio < INTERVALO_THINGSPEAK) return;
```

```
  ultimoEnvio = millis();
  if (WiFi.status() != WL_CONNECTED) return;

  String url = "http://api.thingspeak.com/update?api_key=";
  url += THINGSPEAK_CHAVE;
  adicionarCampo(url, 1, atual.tBme, atual.bmeOk, 1);
  adicionarCampo(url, 2, atual.uBme, atual.bmeOk, 1);
  adicionarCampo(url, 3, atual.pressao, atual.bmeOk, 1);
  adicionarCampo(url, 4, atual.pm1, atual.pmsOk, 0);
  adicionarCampo(url, 5, atual.pm25, atual.pmsOk, 0);
  adicionarCampo(url, 6, atual.pm10, atual.pmsOk, 0);
  adicionarCampo(url, 7, atual.mqVolts, atual.mqOk, 3);
  adicionarCampo(url, 8, atual.uDht, atual.dhtOk, 1);

  HTTPClient http;
  http.setTimeout(4000);
  http.begin(url);
  int codigo = http.GET();
  String resposta = (codigo > 0) ? http.getString() : "";
  http.end();

  // O ThingSpeak responde com o numero da entrada (> 0) quando aceita
  if (codigo == 200 && resposta.toInt() > 0) {
    envioOk++;
    ultimoErroEnvio = "";
  } else {
    envioFalhas++;
    ultimoErroEnvio = (codigo == 200) ? "ThingSpeak recusou (chave errada ou envio rapido demais)"
                                      : "sem resposta do servidor (codigo " + String(codigo) + ")";
    Serial.print("ThingSpeak: falha no envio - ");
    Serial.println(ultimoErroEnvio);
  }
}

void manterWifi() {
  if (strlen(WIFI_NOME) == 0) return;
  if (WiFi.status() == WL_CONNECTED) return;
  if (millis() - ultimaReconexao < INTERVALO_RECONEXAO) return;
  ultimaReconexao = millis();
  Serial.print("Wi-Fi: tentando conectar em \"");
  Serial.print(WIFI_NOME);
  Serial.println("\"...");
  WiFi.disconnect();
  WiFi.begin(WIFI_NOME, WIFI_SENHA);
}

// ============================================================================
// Pagina web
// ============================================================================
String numeroJson(float v, int casas) {
  if (isnan(v)) return "null";
  return String(v, casas);
}

void responderDados() {
  String j = "{";
  j += "\"tBme\":"    + (atual.bmeOk ? numeroJson(atual.tBme, 1) : String("null")) + ",";
  j += "\"uBme\":"    + (atual.bmeOk ? numeroJson(atual.uBme, 1) : String("null")) + ",";
  j += "\"pressao\":" + (atual.bmeOk ? numeroJson(atual.pressao, 1) : String("null")) + ",";
  j += "\"tDht\":"    + (atual.dhtOk ? numeroJson(atual.tDht, 1) : String("null")) + ",";
  j += "\"uDht\":"    + (atual.dhtOk ? numeroJson(atual.uDht, 1) : String("null")) + ",";
  j += "\"pm1\":"     + (atual.pmsOk ? String(atual.pm1)  : String("null")) + ",";
  j += "\"pm25\":"    + (atual.pmsOk ? String(atual.pm25) : String("null")) + ",";
  j += "\"pm10\":"    + (atual.pmsOk ? String(atual.pm10) : String("null")) + ",";
  j += "\"qualidade\":\"" + String(atual.pmsOk ? classificarPm25(atual.pm25) : "---") + "\",";
```

```
  j += "\"mq\":"      + (atual.mqOk ? numeroJson(atual.mqVolts, 2) : String("null")) + ",";
  j += "\"mqAquecendo\":" + String(atual.mqAquecendo ? "true" : "false") + ",";
  j += "\"ok\":{\"bme\":" + String(atual.bmeOk ? "true" : "false");
  j += ",\"dht\":" + String(atual.dhtOk ? "true" : "false");
  j += ",\"pms\":" + String(atual.pmsOk ? "true" : "false");
  j += ",\"mq\":"  + String(atual.mqOk ? "true" : "false") + "},";
  j += "\"ligadoSeg\":" + String(millis() / 1000) + ",";
  j += "\"internet\":\"";
  if (strlen(WIFI_NOME) == 0) j += "nao configurada";
  else if (WiFi.status() == WL_CONNECTED) j += "conectada";
  else j += "desconectada";
  j += "\",\"thingspeak\":\"";
  if (strlen(THINGSPEAK_CHAVE) == 0) j += "nao configurado";
  else {
    j += String(envioOk) + " envios ok, " + String(envioFalhas) + " falhas";
    if (ultimoErroEnvio.length() > 0) j += " (ultimo erro: " + ultimoErroEnvio + ")";
  }
  j += "\"}";
  servidor.send(200, "application/json", j);
}

void responderHistorico() {
  String j = "{\"agora\":" + String(millis() / 1000) + ",\"pontos\":[";
  for (int i = 0; i < histQtd; i++) {
    const Ponto &p = historico[(histInicio + i) % TAM_HISTORICO];
    if (i > 0) j += ",";
    j += "[" + String(p.seg) + "," + numeroJson(p.pm25, 0) + "," + numeroJson(p.tBme, 1) + ","
         + numeroJson(p.uBme, 1) + "," + numeroJson(p.mq, 2) + "]";
  }
  j += "]}";
  servidor.send(200, "application/json", j);
}

void responderCsv() {
  String c = "segundos_ligado,pm2_5_ugm3,temperatura_C,umidade_pct,mq135_V\n";
  for (int i = 0; i < histQtd; i++) {
    const Ponto &p = historico[(histInicio + i) % TAM_HISTORICO];
    c += String(p.seg) + "," + (isnan(p.pm25) ? String("") : String(p.pm25, 0)) + ","
         + (isnan(p.tBme) ? String("") : String(p.tBme, 1)) + ","
         + (isnan(p.uBme) ? String("") : String(p.uBme, 1)) + ","
         + (isnan(p.mq) ? String("") : String(p.mq, 2)) + "\n";
  }
  servidor.sendHeader("Content-Disposition", "attachment; filename=estacao.csv");
  servidor.send(200, "text/csv", c);
}

const char PAGINA[] PROGMEM = R"HTML(<!DOCTYPE html>
<html lang="pt-BR"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Estacao de Qualidade do Ar</title>
<style>
body{font-family:system-ui,sans-serif;margin:0;background:#f4f6f8;color:#1d2733}
header{background:#1f6f5c;color:#fff;padding:14px 16px}
header h1{margin:0;font-size:1.2em}
header small{opacity:.85}
main{padding:12px;max-width:900px;margin:auto}
.grade{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:10px}
.cartao{background:#fff;border-radius:10px;padding:12px;box-shadow:0 1px 3px
    rgba(0,0,0,.08)}
.rotulo{font-size:.8em;color:#5b6b7a}
.valor{font-size:1.7em;font-weight:600;margin-top:4px}
.sub{font-size:.8em;color:#5b6b7a;margin-top:2px}
#qualidade{font-size:1.3em;font-weight:700;padding:10px;border-radius:10px;text-
    align:center;margin:10px 0;color:#fff;background:#888}
h2{font-size:1em;margin:18px 0 8px}
canvas{width:100%;height:160px;background:#fff;border-radius:10px;box-shadow:0 1px 3px
    rgba(0,0,0,.08)}
```

```
.status{font-size:.85em;line-height:1.6}
.ok{color:#1a7f37}.falha{color:#c62828;font-weight:600}
a{color:#1f6f5c}
</style></head><body>
<header><h1>Estação de Qualidade do Ar</h1><small id="ligado">carregando...</small>
</header>
<main>
<div id="qualidade">---</div>
<div class="grade">
 <div class="cartao"><div class="rotulo">PM2.5</div><div class="valor" id="pm25">--
</div><div class="sub">µg/m³</div></div>
 <div class="cartao"><div class="rotulo">PM10</div><div class="valor" id="pm10">--</div>
<div class="sub">µg/m³ · PM1.0: <span id="pm1">--</span></div></div>
 <div class="cartao"><div class="rotulo">Temperatura</div><div class="valor" id="tBme">-
-</div><div class="sub">°C · DHT22: <span id="tDht">--</span></div></div>
 <div class="cartao"><div class="rotulo">Umidade</div><div class="valor" id="uBme">--
</div><div class="sub">% · DHT22: <span id="uDht">--</span></div></div>
 <div class="cartao"><div class="rotulo">Pressão</div><div class="valor" id="pressao">--
</div><div class="sub">hPa</div></div>
 <div class="cartao"><div class="rotulo">Gases (MQ135)</div><div class="valor" id="mq">-
-</div><div class="sub" id="mqSub">V</div></div>
</div>
<h2>PM2.5 (µg/m³) - últimas 2 horas</h2><canvas id="gPm"></canvas>
<h2>Temperatura (°C)</h2><canvas id="gT"></canvas>
<h2>Umidade (%)</h2><canvas id="gU"></canvas>
<h2>Gases MQ135 (V)</h2><canvas id="gMq"></canvas>
<h2>Situação</h2>
<div class="cartao status" id="status">--</div>
<p><a href="/csv">Baixar histórico (CSV)</a></p>
</main>
<script>
const cores={"BOA":"#2e7d32","MODERADA":"#f9a825","RUIM":"#ef6c00","MUITO
RUIM":"#c62828","PESSIMA":"#6a1b9a"};
function txt(id,v){document.getElementById(id).textContent=(v===null||v===undefined)?"--
":v;}
function st(nome,ok){return nome+': <span class="'+(ok?'ok':'falha')+'">'+
(ok?'ok':'FALHA')+'</span>';}
function tempo(s){const h=Math.floor(s/3600),m=Math.floor(s/60)%60;return h+'h
'+m+'min';}
async function atualizar(){
 try{
  const d=await (await fetch('/dados')).json();
  txt('pm25',d.pm25);txt('pm10',d.pm10);txt('pm1',d.pm1);
  txt('tBme',d.tBme);txt('tDht',d.tDht);txt('uBme',d.uBme);txt('uDht',d.uDht);
  txt('pressao',d.pressao);txt('mq',d.mq);
  txt('mqSub',d.mqAquecendo?'V · aquecendo':'V');
  const q=document.getElementById('qualidade');
  q.textContent='Qualidade do ar:
'+d.qualidade;q.style.background=cores[d.qualidade]||'#888';
  txt('ligado','ligada há '+tempo(d.ligadoSeg));
  document.getElementById('status').innerHTML=st('BME280',d.ok.bme)+' ·
'+st('DHT22',d.ok.dht)+' · '+st('PMS7003',d.ok.pms)+' · '+st('MQ135',d.ok.mq)
   +'<br>Internet: '+d.internet+'<br>ThingSpeak: '+d.thingspeak;
 }catch(e){txt('ligado','sem conexão com a estação');}
}
function grafico(id,pontos,col,cor){
 const
c=document.getElementById(id),w=c.clientWidth,h=c.clientHeight,r=window.devicePixelRatio
||1;
 c.width=w*r;c.height=h*r;const g=c.getContext('2d');g.scale(r,r);g.clearRect(0,0,w,h);
 const v=pontos.filter(p=>p[col]!==null);
 g.fillStyle='#5b6b7a';g.font='11px sans-serif';
 if(v.length<2){g.fillText('aguardando dados (1 ponto a cada 30 s)',10,h/2);return;}
 let mn=Math.min(...v.map(p=>p[col])),mx=Math.max(...v.map(p=>p[col]));
 if(mx-mn<1e-6){mx+=1;mn-=1;}
 const t0=v[0][0],t1=v[v.length-1][0],e=40,dir=8,top=8,base=h-18;
 const X=t=>e+(t-t0)/(t1-t0||1)*(w-e-dir),Y=y=>base-(y-mn)/(mx-mn)*(base-top);
```

```
 g.strokeStyle='#e3e8ee';g.beginPath();g.moveTo(e,top);g.lineTo(e,base);g.lineTo(w-dir,base);g.stroke();
 g.fillText(mx.toFixed(1),2,top+8);g.fillText(mn.toFixed(1),2,base);
 g.fillText('-'+Math.round((t1-t0)/60)+' min',e,h-4);g.fillText('agora',w-dir-30,h-4);
 g.strokeStyle=cor;g.lineWidth=2;g.beginPath();
 v.forEach((p,i)=>{i?g.lineTo(X(p[0]),Y(p[col])):g.moveTo(X(p[0]),Y(p[col]));});g.stroke();
}
async function historico(){
 try{
  const d=await (await fetch('/historico')).json();
  grafico('gPm',d.pontos,1,'#c62828');grafico('gT',d.pontos,2,'#ef6c00');
  grafico('gU',d.pontos,3,'#1565c0');grafico('gMq',d.pontos,4,'#6a1b9a');
 }catch(e){}
}
atualizar();historico();setInterval(atualizar,5000);setInterval(historico,30000);
</script></body></html>)HTML"
;

void responderPagina() {
  servidor.send_P(200, "text/html", PAGINA);
}

// ====================================================================
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println();
  Serial.println("=== Estacao de Qualidade do Ar - VERSAO FINAL ===");

  Wire.begin(PINO_SDA, PINO_SCL);
  if (bme.begin(0x76, &Wire) || bme.begin(0x77, &Wire)) {
    bmeIniciado = true;
    Serial.println("BME280  : iniciado");
  } else {
    Serial.println("BME280  : NAO encontrado (confira SDA/SCL/VIN/GND)");
  }

  dht.begin();
  Serial.println("DHT22   : iniciado");

  Serial2.begin(9600, SERIAL_8N1, PMS_RX, PMS_TX);
  Serial.println("PMS7003 : porta serial iniciada");

  analogReadResolution(12);
  analogSetPinAttenuation(PINO_MQ135, ADC_11db);
  Serial.println("MQ135   : entrada analogica iniciada");

  // Wi-Fi: rede propria sempre; rede com internet se configurada
  if (strlen(WIFI_NOME) > 0) {
    WiFi.mode(WIFI_AP_STA);
  } else {
    WiFi.mode(WIFI_AP);
  }
  WiFi.softAP(AP_NOME, AP_SENHA);
  Serial.print("Wi-Fi   : rede propria \"");
  Serial.print(AP_NOME);
  Serial.print("\" criada. Pagina em http://");
  Serial.println(WiFi.softAPIP().toString());

  if (strlen(WIFI_NOME) > 0) {
    WiFi.setAutoReconnect(true);
    WiFi.begin(WIFI_NOME, WIFI_SENHA);
    ultimaReconexao = millis();
    Serial.print("Wi-Fi   : conectando em \"");
    Serial.print(WIFI_NOME);
    Serial.println("\" para enviar ao ThingSpeak...");
```

```
  } else {
    Serial.println("Wi-Fi   : rede com internet nao configurada (ThingSpeak desligado)");
  }

  servidor.on("/", responderPagina);
  servidor.on("/dados", responderDados);
  servidor.on("/historico", responderHistorico);
  servidor.on("/csv", responderCsv);
  servidor.begin();
  Serial.println("Pagina  : servidor iniciado");

  Serial.println("Primeiro resumo em 5 segundos...");
  ultimoRelatorio = millis();
  ultimoHistorico = millis();
}

void loop() {
  servidor.handleClient();
  processarPms();
  processarDht();
  processarMq();

  if (millis() - ultimoRelatorio >= INTERVALO_RELATORIO) {
    ultimoRelatorio = millis();
    atualizarLeituras();
    imprimirRelatorio();
  }

  if (millis() - ultimoHistorico >= INTERVALO_HISTORICO) {
    ultimoHistorico = millis();
    guardarHistorico();
  }

  manterWifi();
  enviarThingSpeak();
}
```

O programa da Etapa 3 está na Listagem 1 (seção 4.1).

Apêndice B – Comandos de terminal utilizados

```
# Fechar a Arduino IDE travada
pkill -f arduino            # ou, forçado:  pkill -9 -f arduino
pgrep -fl arduino           # vazio = fechada

# Remover um pacote esp32 instalado pela metade (IDE 1.8 em snap)
ls ~/snap/arduino/current/.arduino15/packages/esp32/hardware/esp32/
rm -rf ~/snap/arduino/current/.arduino15/packages/esp32
rm -rf ~/snap/arduino/current/.arduino15/staging/packages

# Identificar a placa e a porta
lsusb                       # QinHeng Electronics ... 1a86:55d4
ls -l /dev/ttyACM* /dev/ttyUSB*
groups

# Diagnóstico de permissão
getfacl /dev/ttyACM0
snap connections arduino | grep -i usb

# Liberação temporária (até desconectar a placa)
sudo chmod 666 /dev/ttyACM0

# Liberação permanente
sudo usermod -aG dialout $USER          # vale após sair e entrar na sessão
echo 'SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="55d4", MODE="0666"' | sudo
tee /etc/udev/rules.d/99-esp32.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Apêndice C – Lista de verificações elétricas

Tabela 22 – Verificações com o multímetro antes de cada energização.

Quando	Modo	Pontas	Resultado exigido
Sempre	continuidade	3V3 – GND	**não** apita
Sempre	continuidade	VIN – GND	**não** apita
Etapa 4	continuidade	DHT22 pino 2 – GPIO4	apita
Etapa 4	continuidade	DHT22 pino 4 – GND	apita
Etapa 5	continuidade	SDA – GPIO21 e SCL – GPIO22	apitam
Etapa 6	continuidade	VCC da adaptadora – VIN; GND – GND	apitam
Etapa 7	continuidade	AO do MQ135 – GPIO34	**não** apita
Etapa 7	tensão CC (20 V), com energia	ponto médio – GND	< 2,5 V (projeto: ~0,5 V)
Etapa 7	tensão CC (20 V), com energia	AO – GND	≈ 2 × ponto médio
Etapa 10 (fonte no VIN)	tensão CC (20 V)	saída da fonte	+4,8 a +5,3 V, polaridade conferida
Etapa 10	tensão CC (20 V), tudo ligado	VIN – GND	4,6 a 5,3 V

Fonte: Elaborado pelos autores.